

Rows under Refinement: Branch-Local GADT Equations for Extensible Records

Row polymorphism and GADTs make opposite demands on the same variable: unification solves an equation globally; GADT refinement keeps it branch-local. No published type system refines a row variable by a branch-local GADT equation; the one widely used language that ships both documents the interaction as “not well specified” and restricted its first GADT implementation. We present $\lambda\rho G$, a calculus of branch-local row refinement: a scoped equation store *captures*, rather than solves, the equations a match puts on rigid variables, and a two-tier result rule, **directed by the kind of the equation**, lets a *type* equation coerce at a branch result while a *row* equation never escapes — its discharge would move the row’s domain out of the branch. The discipline yields an operation-level answer: selection is licensed exactly by the heads of the branch’s equations; scoped-label update is domain-preserving, hence sound by construction; insertion, the shape-changing dual, is rejected — a deliberate position on Rémy’s extension dial, not a discovery. A measured principality boundary carries an inversion: the witness encoding *infers* where the native refinement is *checkable*, *not inferable*. An Elm-family checker implements it: the pre-existing corpus compiles byte-identically against a committed manifest, 152 regression checks pass, and an external state machine translated from an OCaml thread now has its impossible match arms refuted at compile time. An adversarial hunt across three passes found five unsound accepts — two in the first escape check, two in the shipping checks, one in the coercion surface — all fixed and pinned by gate fixtures, the two escape-check accepts additionally mechanized as theorems, the last closed by a class-level repair of the equation lifecycle; and the compiler’s own effect interpreter checks 16 of 20 branches honestly, the four holdouts named and decomposed by kind of lie. Ninety Lean theorems (three explicit axioms) mechanize the row-rewrite commutation under duplicate labels — one of them corrects our own specification — the escape domain rule, the update theorem, and the typing judgment’s row-specific obligations (the store discipline and record-operation preservation), with the unifier abstracted as a parameter. We do not prove general soundness: progress and preservation are argued.

Draft, compiled section by section. Target venue: ICFP full paper, 25 pp ACM Small excluding bibliography. Companion material: the calculus specification (docs/research/row-gadt-calculus.md, $\lambda\rho G$), the mechanization (lean/, one Lake project), the reference implementation (elm-compiler/src/Type/), the fixture matrix (tests/elm-fixtures/run-elm-gate.sh), the reproducibility chain (tools/osier-numbers.sh, which reprints every number this paper cites), and the survey (docs/research/osier-related-work.md, which is the bibliography’s source).

1 INTRODUCTION

Extensible records with row polymorphism and GADTs with local type refinement are each well understood. Row polymorphism [Rémy 1994; Gaster & Jones 1996; Leijen 2005] gives records open types — $\{ n : \text{Int} \mid \rho \}$ is a record with an n field and an arbitrary tail ρ — and unification over row variables preserves principal types. GADTs give pattern matches that *refine* type variables branch-locally: matching $\text{IntLit} : \text{Expr Int}$ inside $\text{eval} : \text{Expr } a \rightarrow a$ learns $a \doteq \text{Int}$ for that branch alone. The two mechanisms make opposite demands on the same object. Row unification wants a type variable to be *solvable*: an equation such as $\rho \doteq \{ x : \text{Int} \mid \rho' \}$ is written back into a single global substitution. GADT refinement wants an equation to be *local*: two branches of one match may learn conflicting facts about one variable, so no global substitution can hold them both. Every GADT-inference framework resolves this by making refined variables rigid — untouchable by the solver [Vytiniotis et al. 2011; Pottier & Régis-Gianas 2006] — and every row system resolves it by never putting a row variable under an equation at all. The interaction is

genuinely unoccupied: no published type system refines a *row variable* by a branch-local GADT equation, and no published system therefore says which record operations remain typable when such an equation is active.

The gap is documented, not merely empty. OCaml ships all three features — GADTs, row-polymorphic objects and polymorphic variants, and the locally abstract type prefix `type a.` — and its maintainers describe the interaction precisely. OCaml’s first GADT implementation *restricted* the combination outright: “we had to restrict the use of object types and polymorphic variants in combination with GADTs, to prevent local equations from breaking the invariant that the same row variable may only appear in two record types that are equal” [Garrigue & Rémy 2012]. Its residue is still felt as an undocumented idiom with empirical failure modes: “the interaction of row variables and GADTs is not well specified. Thus it is common to hit hard to understand behaviour, and nothing is guaranteed beyond the fact that the currently implemented interaction is safe” [octachron, OCaml maintainer, Jan 2024]. Pattern matching on a polymorphic variant blocks refinement entirely (“GADT equations cannot narrow a polymorphic variant constraint” [octachron 2024]; `ocaml/ocaml #5724` records an earlier, stronger 4.00-era restriction — no type propagation in any match containing a poly-variant pattern — lifted in 4.01 [Garrigue, rev 13221; 4.01 Changes]), and the recommended workaround restructures the program so that no single row variable is ever refined: model each field as its own one-field object type, which is a manual membership-witness encoding. Practitioners reach for witness encodings by necessity. The one system that met the interaction treated the conflict as unsoundness and legislated around it; the dominant GADT-inference framework cannot even state it — `OutsideIn(X)`’s type grammar $\tau ::= tv \mid Int \mid Bool \mid [\tau] \mid T \tau$ has no record type, no row variable, and no presence [Vytiniotis et al. 2011].

The contrast this paper turns on: where GADT inference decides solvability by *scope* — a touchable/untouchable variable set — $\lambda\rho G$ decides discharge by the *kind* of the equation. A type equation coerces at a branch result; a row equation never escapes, because it carries a *domain* that `OutsideIn(X)`’s type grammar has no room for.

1.1 The motivating example

The smallest program that needs all three features — rows, GADT refinement, and locally abstract types — is a type-indexed row-membership witness (Figure 1). `Has l t ρ` is a GADT proving that the row `ρ` carries a field `l : t`; `select` reads that field through the witness; `setx` replaces it.

```

type Has l t rho
  = Here : Has l t { l : t | rho }
  | There : Has l t rho -> Has l t { k : s | rho }

select : type rho l t. { rho | l : t } -> Has l t rho -> t
select r h =
  case h of
    Here ->
      r.l
    There rest ->
      select r rest

setx : type rho l t. { rho | l : t } -> Has l t rho -> t -> { rho | l : t }
setx r h v =
  case h of

```

```

Here ->
  { r | l = v }

There rest ->
  setx r rest v

```

Figure 1. The central example – the `Has` witness and `select`, verbatim from gate fixture `tests/elm-fixtures/rowgadt_select.elm` (lines 8–20), and `setx`, verbatim from `tests/elm-fixtures/rowgadt_setx.elm` (lines 13–20; the `Has` block is identical in both fixtures); both fixtures compile clean.

Each of the three features is forced:

- **Rows.** $\{ \text{rho} \mid l : t \}$ is an open record; the function must work for every tail ρ , so the record’s type is a row variable.
- **GADT refinement.** Matching `Here` learns the equation $\rho \doteq \{ l : t \mid \rho_1 \}$ – which is exactly what makes $r.l$ well-typed – and matching `There` learns $\rho \doteq \{ k : s \mid \rho_2 \}$ for labels $k \neq l$. The readable field set must *grow inside a branch* and shrink back at its end.
- **Locally abstract types.** The two branches impose *conflicting global* substitutions on ρ . A single substitution cannot hold $\rho \doteq \{ l : t \mid \rho_1 \}$ and $\rho \doteq \{ k : s \mid \rho_2 \}$ for distinct l, k . Only a rigid ρ – abstract within the body, as the type `rho l t.` prefix declares – keeps each branch’s equation local to that branch.

Without locally-abstract rigidity the two branches conflict and `select` has no type – this is the necessity of the third feature. (The *measured* rejection is the **signatureless** form, gate fixture `rowgadt_l3ii`, which reports `cannot unify {k:a| b} with {| a}`; note it measures signaturelessness wholesale rather than a flexible ρ in isolation, since that fixture carries no signature at all.) Without refinement capture the `Here` branch’s equation has nowhere to live; without rows the record cannot stay open.

1.2 The design in one paragraph

$\lambda\rho G$ (Section 2) is a calculus of **branch-local row refinement**. A scoped equation store Δ accompanies the judgment $\Gamma ; \Delta ; R \vdash e : t$, where R is the set of rigid (skolem) variables. At a pattern match, the store is *snapshotted*; the pattern-vs-scrutinee unification runs in *branch mode*, where a unification that would bind a rigid variable is **captured** as an equation in Δ rather than solved – the global substitution θ gains nothing; at branch end the store is *truncated* to the snapshot. Inside a branch, the equations *license* operations: selection $r.l$ is licensed exactly when some equation on ρ exposes l at its head; update $\{ r \mid l = v \}$ is licensed the same way and, crucially, is **domain-preserving** – scoped-label update replaces the first occurrence of l and never extends the row – so the branch result is well-typed at the abstract ρ with *no discharge at all*. At the branch result, a **two-tier** rule decides what may leave the branch: a *type* equation ($a \doteq \text{Int}$) is discharged by a one-way coercion of the re-check – the canonical GADT evaluator type-checks – but a *row* equation ($\rho \doteq \{ l : t \mid \rho' \}$) never discharges, because that would move the row’s *domain* out of the branch; its escape is an error. Insertion, the shape-changing dual of update, never consults the store and is rejected under an active equation.

1.3 The heterogeneous container

The same discipline unlocks type witnesses – the encoding practitioners use to recover an element type a container does not mention. type `Witness a = WInt : Witness Int | WString : Witness String`, an existential wrapper `Some : Witness a -> a -> Any`, and a fold over a list `[Some WInt 3, Some WString "hi"]` all type-check (gate

fixture `rowgadt_het`, run "hi3"): the existential `a` bound by the `Some` pattern is instantiated *rigid* (the “existentials are skolems” rule every GADT system needs; before it, the `a` was a flexible unification variable, a nested match bound it globally, and the sibling `WString` branch conflicted — two independently measured failures), and each nested branch captures `a ≐ Int / a ≐ String` locally, discharging at the use of the payload. This is the feature the witness program generalizes: the *witness encoding* of row membership is what OCaml’s own workaround constructs by hand, and the calculus that makes `Has` typable also makes the hand encoding unnecessary.

1.4 An external program, not a self-referential one

The strongest objection to a paper like this is that its only non-toy evidence is the authors’ own compiler. We answer with a program written by someone else. A `discuss.ocaml.org` thread (“Unable to refute impossible GADT pattern with polymorphic variants”, t/13718) models a door state machine: states as types, transitions as a GADT-witnessed relation, and a fold over the chain of events — and there the author hits the wall this paper characterizes, on all three sides: illegal construction is rejected, but the *impossible arms* of the chain match cannot be refuted, and the recommended fix is the per-field-object workaround. We retrieved and read the thread in full and translated the program into the reference implementation — the surface of $\lambda\rho G$ (the example lives at `examples/research/fsm/door.elm`; gate fixtures `rowgadt_fsm`, `rowgadt_fsm_bad`, `rowgadt_fsm_narrow`, `rowgadt_fsm_nested`, `rowgadt_fsm_nested_bad` pin its behaviour).

States are rows — `Locked = {}`, `Closed = { closed : Int }`, `Open = { open : Int }`, `Broken = { broken : String }`. A state is its marker plus a shared counter `n : Int`; the alias `State marker = { marker | n : Int }` names that row extension:

```
type alias State marker =
  { marker | n : Int }

type Event from to
  = Unlock : Event {} { closed : Int }
  | Open   : Event { closed : Int } { open : Int }
  | Close  : Event { open : Int } { closed : Int }
  | Lock   : Event { closed : Int } {}
  | Jiggle : Event { closed : Int } { closed : Int }
  | Break  : Event { open : Int } { broken : String }
  | Reset  : Event { broken : String } {}
```

```
readFrom : type from to. State from -> Event from to -> String
readFrom rec ev =
  case ev of
    Unlock -> "locked#" ++ String.fromInt rec.n
    Open   -> "closed:" ++ String.fromInt rec.closed
    Close  -> "open:"   ++ String.fromInt rec.open
    ...
```

Where OCaml’s equations “cannot narrow a polymorphic variant constraint”, the reference implementation’s narrowing is direct and *per-branch*: in the `Open` branch the from-row is refined to `{ closed : Int }` and `rec.closed` is well-typed *only there* — in the `Close` branch (from-row `{ open : Int }`) the read `rec.broken` is rejected with type variable `a` is rigid (from the signature) and cannot be unified with `{broken:a}`

b} (gate fixture `rowgadt_fsm_narrow`, re-measured), while the *same field* is legal in the `Reset` branch (from-row `{ broken : String }`): the rejection is per-branch, not per-field. No per-field restructuring is needed: the states *are* rows, and the refinement is the mechanism, not the workaround.

The thread’s *headline* wall is refutation, and that too is answered. The source author could not get OCaml to refute the impossible arms of the chain match (its `->` . mechanism). The reference implementation now checks *exhaustiveness* — a match omitting a possible arm is a compile error, not a runtime raise (fixture `adtgaps`) — and *refutes* impossible arms under the branch equations: `describeBroken : Step { broken : String } -> String` matching only `Then` `prev Break` compiles clean, because `Start : Step {}` and every other event are impossible at that index (fixture `refutpos`; the negative `refutneg` pins that a *possible* arm, `HNil` at an open `HList p`, is not refuted and must still be matched). Refutation has a stated condition: it fires only where the scrutinee’s index is *concrete or binder-rigid*; a bare flexible index is never concretized to refute a sibling (fixture `refutbare` — a partial `f : Tag a -> Int` matching only one arm correctly errors `non-exhaustive case: missing B`).

The translation is honest about three boundaries, stated here rather than hidden. (i) The transition GADT is *row-inert*: `Event`’s indices are ordinary type parameters, so the row refinement exercised is on the *record argument*’s row variable — the honest claim is “an external FSM translated to rows where per-state field reads use branch-local row refinement”, not “the FSM expressed directly”. (ii) A reducer whose *result* is a record at the refined row is rejected by the escape rule — including the state-preserving *Jiggle* update, because the refined variable there is the result index; the FSM keeps its state type-level, as the OCaml original does. (iii) The membership-witness reducer `read` is definable but not *callable* at a concrete witness (`read Here { n = 1, closed = 2 }` errs `cannot unify a with {l:a| b}` — a global bare-row-variable gap in the implementation, outside branch mode); the example keeps the witness as the shape `read From` reduces to, not as a working reducer. The example demonstrates capability where it holds and names the boundary where it does not.

1.5 Why the cell stayed empty

The empty cell is not an accident of oversight. Rows and GADTs were adopted by disjoint language populations — ML-family record calculi in the early 1990s, GADT inference in the mid-2000s — and the frameworks that made GADT inference tractable (`OutsideIn(X)`, stratified inference, `HMG(X)`) were built on type grammars without rows. Where the two populations met, rows were *secondary*: in OCaml, objects and polymorphic variants are not the primary record mechanism, so restricting their interaction with GADTs costs little and the per-field-object workaround is cheap. In a language where rows are the primary record mechanism, the same restriction removes the core feature. And the interaction is a *kinding* problem as much as an equation problem: three of the four implementation blockers we hit building the reference implementation were kinding defects (Section 6) — a calculus that does not make row-variable kinding explicit cannot predict where a real implementation breaks. Generic local-assumption frameworks could in principle host this: `HMG(X)` is parameterised over a constraint domain `X`, but no published instantiation of it targets a row/record theory, and the two row-specific obligations our discipline needs (the rewrite/equation commutation under scoped-label duplicates, and the domain escape rule) do not come out of the generic frame for free (Section 4).

1.6 Contributions

This paper’s claims are of three kinds — *mechanized* (machine-checked in the Lean development), *measured* (reproduced by the reference implementation, with fixtures registered in a regression gate), and *argued* (stated with their mechanism, not proved) — and the text marks each. In outline:

1. **A discipline for an unoccupied cell, built on one principle.** $\lambda\rho G$ is the first designed account of what happens when a row variable sits under a branch-local GADT equation. The principle is a discharge rule **directed by the kind of the equation**: at a branch result a *type* equation coerces, while a *row* equation never discharges — its discharge would move the row’s domain out of the branch. The machinery is a scoped equation store that snapshots, captures would-be rigid bindings as equations, and truncates on branch exit (Sections 2–3). The program of Figure 1 type-checks under the discipline; its signatureless form is rejected — the measured face of the rigidity requirement.
2. **An operation-level answer, with the positive row result mechanized.** Under an active refinement: selection is licensed exactly by the heads of the branch’s equations (a field in no head is an error inside the branch); update is sound *by construction*, because scoped-label update is domain-preserving and the branch result is well-typed at the abstract row with no discharge; insertion is rejected — a deliberate position on a dial Rémy set to *unconditional* and Gaster & Jones to *strict*, not a discovery (Section 3; mechanized: `update_domain_preserving`, `update_reflexive_domain`, `update_accepted_at_rho`, `insertion_rejected_under_refinement`).
3. **A measured principality boundary, and its inversion.** Principality is retained where every refinement’s target is a branch-scoped existential — the *witness encoding infers* with no signature — and lost exactly where two sibling branches refine one *signature* row variable whose result mentions it: there the native form is *checkable*, *not inferable*. The inversion is the result: the encoding does not merely model rows, it **moves the boundary to the other side** of the native form (Section 5, the L3 measurement).
4. **Mechanized metatheory for the row obligations, validated by an adversarial hunt.** Ninety machine-checked Lean theorems (three explicit axiom parameters; clean build, zero `SORRY`) cover the row-rewrite commutation under duplicate labels — including the theorem that *corrected our own specification* (`h1_head_only_vs_full`: the “for every label” form holds only in the rigid-tail regime) — the escape rule as a *domain* rule (exact for fresh labels, coarse for duplicates), the update theorems, and both escape counterexamples. An adversarial hunt across three passes found five *real unsound accepts* — the first escape check’s let-laundering through let-generalization and its wildcard sibling leak, two in the shipping checks, and one in the coercion surface — all fixed, all pinned by fixtures; the two escape-check accepts are additionally mechanized as theorems (`H2b_no_quantify_refined_tail`, `H2b_wildcard_leak_shape`), and the coercion-surface one is closed by a class-level repair of the equation lifecycle (Section 6.2).
5. **A working reference implementation, measured against itself and an external program.** The discipline runs inside an Elm-family checker: the pre-existing corpus compiles *byte-identically* after the change — 149 artifacts with zero moved — and the committed sha256 manifest holds 149 entries; the gate passes 152/152, and the compiler’s own effect interpreter checks 16 of its 20 branches honestly (reproducible bisection script; the four holdouts named, one of them deliberate) — (Section 6). The external program — a state machine translated from an OCaml discussion thread — reads per-state fields under branch-local refinement and has its impossible match arms **refuted at compile time**, the thread’s headline wall answered under the rigidity guard of Section 1.4.

Scope, stated once. We do not prove general soundness: progress and preservation are argued, not proved. What the mechanization *does* prove are exactly the row-specific obligations the argued soundness rests on — the rewrite/equation commutation (H1), the update domain theorem, the store discipline and escape rule of the typing judgment (`tier_R_forces_escape` and companions), and record-operation preservation — plus the escape counterexamples, all with the unifier abstracted as a parameter (Section 4.1). A paper claim we cannot trace to a theorem name, a fixture, or a measurement is cut rather than hedged.

1.7 The map of the paper

Section 2 presents λpG : syntax with explicit row kinds, the two unification modes, and the rule set. Section 3 gives the two-tier result rule and the operation-level answer: which record operations survive an active refinement. Section 4 states the metatheory — mechanized, argued, and refuted — and the negative result that unrestricted global row refinement is unsound. Section 5 locates the principality boundary and its inversion. Section 6 reports the implementation, the measured correspondence, and the adversarial hunt. Section 7 organizes related work by the feature-pair map — the organization itself is a contribution, because it makes the empty cell visible. Section 8 states limitations; Section 9 concludes.

2 APG: A CALCULUS OF BRANCH-LOCAL ROW REFINEMENT

This section presents the declarative calculus; the full rule set with its implementation seats is the companion specification (`docs/research/row-gadt-calculus.md`), and the mechanization of its hard lemmas is Section 4. Here we give syntax, the judgment, the two unification modes, the store discipline, and the rule inventory (Figures 2 and 3).

2.1 Syntax and kinds

k	$::= \text{Ty} \mid \text{Row}$	kinds
$\hat{x}$	$::= (\text{id}, k, \phi)$	variables: id, kind, flex marker
ϕ	$::= \text{none} \mid \text{number} \mid \text{comparable} \mid \text{appendable}$	
t	$::= a \mid T \hat{t} \mid t \rightarrow t \mid (\hat{t}) \mid \{ r \}$	types
r	$::= \varepsilon \mid \ell : t \mid r \mid \rho$	rows: fields in source order + tail
s	$::= \forall B; \hat{x}. t$	schemes with a bound subset B

Figure 2. Syntax and kinds — kinds, variables with flex markers, types and rows, and schemes with a bound subset. Source: `docs/research/row-gadt-calculus.md` §1.1–1.4 (compressed).

Labels ℓ are surface strings (there are no type-level labels). Duplicate labels are legal and retained; the **first occurrence** of a label is the one selection, restriction, and update act on (Leijen’s scoped labels) — this is definitional and load-bearing: the duplicate case is exactly where the hard lemma bites (Section 4.2). Substitutions θ are kinded and id-keyed; a **ROW** variable’s image is always a record value (a row variable never maps to a bare type).

Kinds are explicit, and the build proved they must be (Section 6.1): three of the four implementation blockers were kinding defects. Well-formedness keeps a **ROW** variable only in tail position (by construction: the row grammar’s tail is $\varepsilon \mid \rho$, never a type variable).

Declarations carry per-constructor **annotated result types** — the GADT form:

```
data Has (l : Ty) (t : Ty) (p : Row) where
  Here  : Has l t { l : t | p' }      -- p' a fresh per-ctor existential
  There : Has l t p0 -> Has l t { k : s | p0 }
```

A constructor’s scheme is $\forall$ its free variables; the variables *not free in its result* are its true existentials.

2.2 The judgment and the store

$\Gamma ; \Delta ; R \vdash e : t$

Figure 2 (continued). The judgment $\Gamma ; \Delta ; R \vdash e : t$. Source: docs/research/row-gadt-calculus.md §2.1.

Γ types term variables (with schemes); Δ is the **equation store** — a most-recent-first list of equations $\hat{x} \doteq t$ (either kind, kind-matched); R is the **set of rigid variable ids** (skolems). The **global substitution** θ — the persistent state of inference — is deliberately *separate*: equations in Δ are never solved into θ while their target is rigid.

Rigidity has exactly three scoped origins: **signature-bound** (the type $\hat{x}$. prefix, or a GADT-index position — Section 6.3); **constructor-existential** (a pattern’s constructor quantifiers not free in its result are skolems of that branch — OutsideIn’s touchables, row-aware); and **index-lift** (a signature index free in the scrutinee that a pattern’s result equates to a structured type is lifted rigid for the branch’s duration). Everything else is flex.

2.3 Unification’s two modes

$\text{unify}_{\delta}(\theta, R; t_1 \doteq t_2)$ with $\delta \in \{\text{global}, \text{branch}\}$ is ordinary unification (decomposition, occurs check, scoped-label row rewrite) plus one discipline: what happens when unification *would* bind a rigid variable.

- **U-RIGID-BIND.** $a \doteq t$ with a rigid: **global** fails (RigidVar); **branch** captures the equation $a \doteq t$ into Δ and leaves a unbound.
- **U-RIGID-BIND-R.** $\rho \doteq \{r\}$ with ρ rigid: same split — with one exception, the **alias case**: if the row is a bare flex variable ($\rho \doteq \rho'$, no fields), the flex variable is bound to $\{\epsilon \mid \rho\}$ in both modes, and nothing is captured. (The dual direction — rigid bound to flex — never arises: binding direction is always flex-gets-bound.)
- **U-RIGID-EXTEND.** The row rewrite reaching a rigid tail it would extend: **global** fails; **branch** captures $\rho \doteq \{\ell : \gamma \mid \beta\}$ (fresh γ, β) and *proceeds*, handing out the exposed field view — the one branch-mode behaviour that continues with a derived fact rather than failing.

What a rigid row tail may be — the inventory, stated once. A rigid ρ **may**: (a) be aliased by a flex row variable; (b) be *locally known*, inside a branch, to have a shape $\{\ell : t \mid \rho'\}$ — a captured equation that licenses selection/update discharge within that branch only; (c) unify with itself. A rigid ρ **may never — in any mode — be bound in θ** : not to a closed row, not to a fielded row, not to another rigid variable outside branch mode, and no rewrite may extend it outside branch mode. In branch mode a would-be binding is captured, and θ **gains nothing**: this is the invariant that makes the exported scheme honest.

2.4 The rules

The rule set — compressed here; the full rules, verbatim from the companion spec, are Figure 3:

- **R-VAR** instantiates a scheme’s quantifiers with fresh *flex* variables of matching kind.
- **R-TYPE** checks a signed declaration’s clauses against the declared scheme with the type-bound quantifiers rigid; unnamed quantifiers stay flexible, restored by the **too-general police** after the body (Section 6.3).
- **R-APP** unifies function with argument *globally*; on a rigid failure explained by Δ (**R-APP-DISCH**), the *type* equation is applied one-way (zonked both sides, occurs-guarded) and the unification re-run. This is the variable-use discharge: `String.fromInt x` with $x : a$ under $a \doteq \text{Int}$ succeeds without binding a .

- **R-SEL** selects: unify the record with $\{ \ell : a \mid \beta \}$; on a rigid-tail failure explained by Δ (**R-SEL-DISCH**) — some equation on ρ zonks to a row whose *head* is ℓ — the selection is licensed and returns the equation’s exposed type. **An equation whose head label differs from ℓ licenses nothing**: the readable field set inside a refining branch grows exactly by the heads of the branch’s equations (`rowgadt_absentfield`).
- **R-UPD** updates: restrict the first occurrence of ℓ (requiring it present) and re-prepend; when the label is present only through a refinement (**R-UPD-DISCH**), the same discharge licenses it and the result is the discharged shape re-prepended. **Nothing is bound**: the equation is read, never consumed.
- **R-UPD-INS (rejection)**. Insertion $\{ e \mid \ell \leftarrow v \}$ — which may extend or shadow — *never* consults the store; if the base’s row tail is refined by this branch, the form is rejected outright (“insertion under rigid row refinement”).
- **R-RESTR** restricts ($\{ e - \ell \}$): first-occurrence removal, no store consultation.
- **R-CASE** is the discipline’s heart: **snapshot Δ ; unify pattern-vs-scrutinee in branch mode (capture); infer the body under $\Delta \cup \Delta_i$ and the branch’s lifted rigid set; check the body result (R-RESULT); truncate Δ to the snapshot**. Truncation is by construction exact — restore assigns the snapshot prefix — so nesting is safe at any depth. Two *memories* survive into the clause: the refined targets (every variable a branch of this clause refined) and the refined tail pairs (each captured row equation’s head and tail) — they are what the result check enforces (R-RESULT), not a channel by which equations leak.
- **R-RESULT** is the two-tier rule — Section 3.1.
- **R-EXISTS** instantiates a pattern’s constructor scheme with its true existentials rigid (branch-scoped); non-GADT constructors have every quantifier free in the result, so ordinary ADT patterns are unaffected, and flex-marked quantifiers are never rigidified (this is what keeps a `Dict k v` pattern from skolemizing `k` — the prelude stays green).
- **R-LET** generalizes the free variables of the bound type not free in the environment — with the refined-tail exclusion that closes the let-laundrying hole (Section 4.4).

The implementation’s two-pass declaration-directed *retry* is an artifact for corpus byte-identity (Section 6.5); **the declarative calculus has only the declaration-directed rule**.

R-VAR

$$\frac{}{\Gamma(x) = s}$$

$$\Gamma ; \Delta ; R \vdash x : \llbracket s \rrbracket$$

R-TYPE (clause checking)

$$\frac{\Gamma(f) = \forall B; q_j. t \quad B = \{\tilde{x}_1 \dots \tilde{x}_n\} \quad (\text{the `type`-prefixed names})}{\text{instantiatePartial: } q_j \mapsto \text{fresh } \tilde{q}_j; \quad \tilde{q}_j \text{ rigid} \iff q_j \in B \wedge \text{flex}(q_j) = \text{none}} \quad \Gamma ; \Delta ; R \cup \{\tilde{q}_j \mid q_j \in B\} \vdash \text{clauses of } f \text{ against } t[q_j := \tilde{q}_j]$$

R-TYPE-POLICE (annotationNotTooGeneral)

$$\frac{\Gamma(f) = s_{\text{declared}} \quad \text{body inferred at fullType (args} \rightarrow \text{result)}}{s_{\text{body}} = \text{generalize}(\text{zonk}(\text{fullType})) \quad (\forall\text{-closure of } f_v)} \quad \text{skolemize } s_{\text{declared}} \text{ (ALL none-flex quantifiers rigid), flex-instantiate } s_{\text{body}} \quad \text{unify_global}(\text{declared_skolem} \doteq \text{body_flex}) \text{ must SUCCEED} \quad \text{else FAIL: "annotation is too general"}$$

R-APP

$$\frac{\Gamma ; \Delta ; R \vdash e_0 : t_0 \quad \text{peel } t_0 \doteq t_1 \rightarrow t' \quad (t', \text{ fresh result})}{\Gamma ; \Delta ; R \vdash e_1 : t_1' \quad \text{unify_global}(t_1 \doteq t_1') \triangleright \theta'} \quad \text{(on failure: R-APP-DISCH)}$$

R-APP-DISCH

$$\frac{\Gamma ; \Delta ; R \vdash e_0 \quad e_1 : t \quad \text{attempted; unify_global fails RigidVar}(a, t) \quad a \doteq \tau \in \Delta, \quad a : Ty, \quad \tau \text{ not a row, } a \notin \text{fv}(\tau) \quad (\text{occurs-guard})}{\Gamma ; \Delta ; R \vdash e_0 \quad e_1 : \theta'[a := \tau](t)} \quad \text{(one-way re-check, no binding)}$$

R-SEL

$$\frac{\Gamma ; \Delta ; R \vdash e : t \quad \text{fresh } a : Ty, \beta : \text{Row} \quad \text{unify_global}(t \doteq \{ \ell : a \mid \beta \}) \triangleright \dots \quad (\text{on rigid failure: R-SEL-DISCH below})}{\text{R-SEL}}$$

```

 $\Gamma ; \Delta' ; R' \vdash e.l : a$ 

 $\frac{\Gamma ; \Delta ; R \vdash e : \{ r \mid \rho \} \quad \rho \text{ rigid} \quad \rho \doteq \{ \ell : t' \mid \rho'' \} \in \Delta \text{ (zonked once; head label is } \ell \text{)}}{\Gamma ; \Delta ; R \vdash e.l : t'} \text{ R-SEL-DISCH}$ 

 $\frac{\Gamma ; \Delta ; R \vdash e : \{ r \} \quad \text{restrict}(r, \ell) = (t_\ell, r') \text{ (}\ell \text{ present, first occurrence)} \quad \Gamma ; \Delta ; R \vdash v : t_v \quad \text{unify\_global}(t_v \doteq t_\ell) \triangleright \theta'}{\Gamma ; \Delta' ; R' \vdash \{ e \mid \ell = v \} : \{ \ell : \theta'(t_v) \mid r' \}} \text{ R-UPD}$ 

 $\frac{\Gamma ; \Delta ; R \vdash e : \{ \text{known} \mid \rho \} \quad \rho \text{ rigid, restrict}(\text{known}, \ell) \text{ fails} \quad \rho \doteq \{ \ell : t' \mid \rho'' \} \in \Delta \text{ (zonked once)} \quad \Gamma ; \Delta ; R \vdash v : t_v \quad \text{unify\_global}(t_v \doteq t') \triangleright \theta'}{\Gamma ; \Delta ; R \vdash \{ e \mid \ell = v \} : \{ \ell : \theta'(t_v) \mid \text{known} ++ \rho''\text{-fields} \}} \text{ R-UPD-DISCH}$ 

 $\frac{\Gamma ; \Delta ; R \vdash e : \{ \text{known} \mid \rho \} \quad \rho \text{ has an equation in } \Delta}{\text{escaping-error: "insertion under rigid row refinement"}} \text{ R-UPD-INS (REJECT)}$ 

 $\frac{\Gamma ; \Delta ; R \vdash e : t \quad t \text{ zonks to } \{ r \} \text{ with } \text{restrict}(r, \ell) = (t_\ell, r')}{\Gamma ; \Delta ; R \vdash \{ e - \ell \} : \{ r' \}} \text{ R-RESTR}$ 

 $\frac{\Gamma ; \Delta ; R \vdash e_1 : t_1 \quad s = \forall (fv(t_1) - fv(\Gamma) - \text{appendables}). t_1}{\Gamma, x : s ; \Delta ; R \vdash \text{let } x = e_1 \text{ in } e_2 : t_2} \text{ R-LET} \quad (\text{where } \Gamma, x : s ; \Delta ; R \vdash e_2 : t_2)$ 

 $\frac{\begin{array}{l} \Gamma ; \Delta ; R \vdash e : t_s \quad (\text{the scrutinee}) \\ \text{For each branch } i \text{ (pattern } p_i, \text{ body } e_i): \\ \quad \Gamma \vdash p_i : t_p \triangleright \text{binds}_i \quad (\text{pattern inference; ctor arms via R-EXISTS}) \\ \quad \Delta^0 = \Delta \quad (\text{SNAPSHOT the store}) \\ \quad \text{unify\_branch}(t_p \doteq t_s) \triangleright \Delta_i \quad (\text{capture: would-be rigid binds become equations}) \\ \quad \Gamma, \text{binds}_i ; \Delta \cup \Delta_i ; R \cup \text{lifted}(p_i, t_p, t_s) \vdash e_i : t_b \\ \quad \text{branch-result check (§3.1) of } t_b \text{ against } t_r \\ \quad \Delta := \Delta^0 \quad (\text{TRUNCATE: restore the snapshot}) \end{array}}{\Gamma ; \Delta ; R \vdash \text{case } e \text{ of } \{ p_i \mapsto e_i \} : t_r} \text{ R-CASE}$ 

 $\frac{\begin{array}{l} \text{unify\_global}(t_b \doteq t_r) \text{ fails RigidVar}(a, t) \\ a \doteq \tau \in \Delta \text{ with } a : \text{Ty} \text{ (a TYPE equation - } \tau \text{ is not a row)} \\ a \notin fv(\tau) \text{ (occurs-guard)} \end{array}}{\text{re-check: unify\_global}(t_b[a := \tau] \doteq t_r[a := \tau]) \triangleright \theta' \text{ (one-way, both sides)}} \text{ R-RESULT-Tier-T (discharge)}$ 

– on success: branch result ok;  $\tau$ 's equation still dies at branch end  

– on nested rigid failure on  $b \neq a$ : recurse once (same rule)  

– on any other failure: the historical error

 $\frac{\begin{array}{l} C : \forall q_j. t_1 \rightarrow \dots \rightarrow t_n \rightarrow t^c \quad \text{determined} = fv(t^c) \cap \{q_j\} \\ \text{instantiate: } q_j \mapsto \text{fresh } \hat{q}_j; \quad \hat{q}_j \text{ rigid} \iff q_j \notin \text{determined} \wedge \text{flex}(q_j) = \text{none} \end{array}}{\Gamma \vdash C \ p_1 \dots p_n : t^c[q_j := \hat{q}_j] \triangleright \text{binds}} \text{ R-EXISTS}$ 

existentials +=  $\{ \hat{q}_j \text{ rigid, scoped to branch} \}$


```

Figure 3. The typing rules of $\lambda\rho G$, in the order of the inventory above. Source: docs/research/row-gadt-calculus.md §2.4–2.9, verbatim rule blocks (the prose around them, with the implementation seats, remains in the companion spec) — the one departure is R-CASE’s internal cross-reference, repointed from the spec’s §2.7 to this paper’s §3.1, where the branch-result rule lives.

3 THE TWO-TIER RESULT RULE AND THE OPERATION-LEVEL ANSWER

3.1 The two-tier result rule and the operation inventory

At each branch’s result, the body type t_b is unified with the expected t_r . On a rigid failure the decision is two-tier — the asymmetry is the paper’s central design claim:

Tier-T (type equations discharge). If $a \doteq \tau$ is in Δ with $a : \text{Ty}$ and τ not a record type (occurs-guarded), the branch result is re-checked with a one-way replaced by τ on *both* sides — a single pass, never a bind. The equation still dies at branch end. This is the standard GADT coercion: a branch knowing $a \doteq \text{Int}$ may return an Int at type a , so the canonical three-branch

evaluator type-checks (`rowgadt_eval`), and a wrong body — a `String` where the equation says `Int` — still fails the re-check (`rowgadt_evalbad`).

Tier-R (row equations never discharge). A row equation $\rho \doteq \{r\}$ never discharges at a result. The reason is *domains*: discharging it would identify the abstract row with one particular field set, moving a domain change out of the branch — the exported scheme would promise a domain the signature does not. Its rigid failure surfaces as the explicit *escaping-row-equation* error.

The full decision procedure at a branch result, in order (Figure 4):

1. Unify (both sides zonked first).
2. On success, three gates before acceptance, in order: the **tail-escape check** — did this unification newly identify a refined tail with its head? (`dropIntroduced`: the drop shape) $\rightarrow$ reject; the **flex $\rightarrow$ rigid alias gate** — did it alias a refined target? (`refinedTargetAliasedBy`) $\rightarrow$ reject; the **branch-end occurs check** — does a pending `KType` equation’s body now contain its target? (`pendingTypeOccurs`) $\rightarrow$ reject (*infinite type*). All three pass $\rightarrow$ done.
3. On rigid failure: does the presented type *rebuild* the head’s refinement (unifies with the equation body, head not free in it)? $\rightarrow$ **accept without binding** (`rebuildMatches`).
4. Else, is there a usable *type* equation? $\rightarrow$ Tier-T discharge.
5. Else, is the rigid variable this branch’s constructor existential? $\rightarrow$ reject (*escaping existential*).
6. Else, does any equation explain it? $\rightarrow$ reject (*escaping row equation*).
7. Else: the historical rigid error.

Figure 4. The branch-result decision procedure. Source: `docs/research/row-gadt-calculus.md` §2.7 and the code — `unifyResultM` (`elm-compiler/src/Type/Infer.elm:1498`): unify first (:1509), then on success the three gates `dropIntroduced` (:1511), `refinedTargetAliasedBy` (:1515), `pendingTypeOccurs` (:1526); on rigid failure `rebuildMatches` (:1534) and the Tier-T discharge (:1542). Helpers: `tailReachesHead` (:1818), `dropIntroduced` (:1842), `rebuildMatches` (:1879), `refinedTargetAliasedBy` (:1709).

The operation-level answer. The rules decide, for each record operation, whether it survives an active refinement $\rho \doteq \{ \ell : \tau \mid \rho' \}$ on the base’s tail:

operation	under refinement	why
select ℓ (the equation’s head)	licensed	R-SEL-DISCH reads the equation’s exposed type
select $m \notin$ equation heads	rejected	no equation exposes m (<code>rowgadt_absentfield</code>)
update $\ell = v$	licensed, no discharge needed	update is domain-preserving — Section 3.2
insert $\ell \leftarrow v$	rejected	insertion is shape-changing; R-UPD-INS never consults Δ

Selection is *licensed exactly by the heads of the branch’s equations*; update is *sound by construction*; insertion is the shape-changing dual, *rejected conditionally* — a deliberate position on the extension dial, not a discovery (Section 3.3).

3.2 Update is safe by construction

Scoped-label update replaces the **first occurrence** of ℓ and requires it present; it never extends the row. So under the refinement $\rho \doteq \{ \ell : t \mid \rho' \}$:

$$\text{dom}(\{ \ell : t_v \mid \rho' \}) = \text{dom}(\{ \ell : t \mid \rho' \}) = \text{dom}(\rho) \quad \text{under the equation}$$

The branch may return the updated record *at the abstract row* $\{ \rho \}$ — the domains are equal, no equation is consumed, nothing escapes. Update needs no discharge rule at all: its compatibility falls out of its semantics, not out of the type system. The dual makes the claim non-vacuous: insertion *can* change the domain, which is exactly why it may never consult the store. Both directions are mechanized (`update_reflexive_domain`, `update_accepted_at_rho`, `insertion_rejected_under_refinement`; Section 4.5).

3.3 Positioning the insertion trade and the update lineage

Insertion is typable in the rows-only world — this paper’s rejection exists *only relative to an active branch equation*, and the dial has published settings. **Rémy 1994** types unrestricted field extension *unconditionally*: `new_a :  $\Pi(a:\phi;\$) \rightarrow \alpha \rightarrow \Pi(a:\text{pre}(\alpha);\$)$` , no lacks predicate, no presence check; his `with` never requires the field present and imposes “no constraint between the types of the old and the new values”. **Gaster & Jones 1996** dial the same operation to *strict*: $(\ell = |) :: (r \setminus \ell) \Rightarrow \dots$, extension under a lacks predicate. **CORELINKS** types insert by *quantifying presence* over the needed row. Our rule is the fourth position — *conditional*: unrestricted outside refinement, rejected under an active equation — and the first account of *why* the condition exists: a discharged insertion is a domain change escaping its branch. We give up Rémy’s expressiveness exactly where it would break refinement soundness.

Record update has the same lineage, and the contrast is the direction that matters. Rémy’s update is extension (shadowing; the domain may grow); Gaster & Jones’ is restrict-then-extend under lacks; the Links system derives remove-then-extend — attested in the CORELINKS aside, not in Links’ own FMCO’06 paper, which defines no record update. Ours is first-occurrence scoped-label replace — **domain-preserving by construction** — which is what makes update’s discharge free. The novelty is being domain-preserving, not being typable at all.

4 METATHEORY: MECHANIZED, ARGUED, REFUTED

This section separates what is *proved* (machine-checked in the Lean development: 90 theorems, zero `sorry`, three explicit axiom parameters — `Unifies/Captures at Typing.lean:221,229`, the unifier abstracted as a parameter of the theory, and `unifies_field_projection at Preserve.lean:213`; `lake build clean`), what is *measured* (fixtures, re-run in the gate), and what is *argued* (stated with its mechanism, not proved). The negative result is *stated, not proved*; it is the necessity direction, and its mechanism is measured.

4.1 The architecture: unification as a parameter

The typing judgment is mechanized constraint-based (`Typing.lean`): the rules *emit* unification constraints against the abstract relation `Unifies R t1 t2` and capture equations through the abstract `Captures R tp ts  $\Delta_i$` — no unification algorithm appears inside the rules, and no substitution is threaded through them. Soundness of the calculus then requires the unifier to be *sound* (it must not accept inconsistent constraints), not *complete* — which separates the row-specific obligations from the (undecidable-complete) unifier problem. The store discipline is visible in the rule shapes: `HasTypeBranches` threads one snapshot Δ ; each branch body is checked under $\Delta_i \rightarrow \Delta$ with its existentials added to R and its result check run there; the recursion and the conclusion are back under the bare Δ . The two-tier rule is verdict-indexed (`Result  $\Delta$  R tb`

$t_r v$ with $v \in \{\text{ok}, \text{escape}\}$): Tier-T is a productive case (it re-checks and may conclude ok); Tier-R is *terminal* — a needed row equation forces escape. The asymmetry is in the type of the rule, not merely in its premises. Theorems `tier_R_forces_escape` (`TypingStore.lean`) and `tier_T_ok` (`Preserve.lean`) pin both directions.

The assumption, named. The theorems about the typing judgment are conditional on the axiom parameters — stated, not hidden: the store discipline (`TypingStore.lean`) is conditional on `Unifies/Captures` and on nothing else; the preservation results (`Preserve.lean`) are conditional on the unifier being *sound* (it must not accept inconsistent constraints) and, for the plain-selection case, on `unifies_field_projection` (the discharged-selection and update cases need no unifier axiom). The row-algebra theorems (H1, the domain rule, the update theorem, both counterexamples) never mention the axioms. The correspondence between the abstract relation and the implementation’s unifier is *measured*, not proved: the gate’s 152 checks, the approximation inventory of Section 6.5, and the three-way agreement of Section 4.7 stand on the same boundary. Mechanizing the concrete row unifier against these axioms is future work; it would discharge the assumption, not change any theorem’s statement.

4.2 H1 — rewrite/equation commutation, under duplicates

The genuinely novel formal content — no prior system has the setting. A branch’s equation $\rho \triangleq \{ \ell : t \mid \rho' \}$ is *consulted* by discharge, which reads the equation’s head; the row rewrite (scoped-label swap) is the operation that exposes a field of a row. H1 says the two commute, and its hard case is **duplicate labels**: a row may legally carry a duplicate ℓ *before* the equation’s occurrence, in which case the first occurrence — not the equation’s — is what selection must expose.

The five mechanized theorems (`RowGadt.lean`):

- **h1_find_commutes** — first-occurrence search commutes with scoped-label substitution: discharge-then-rewrite and rewrite-then-discharge agree.
- **h1_duplicate_shadow** — a duplicate ℓ before the equation *shadows* it: both routes expose the earlier field, never the equation’s type. (This case was the open probe; the theorem is its confirmation.)
- **h1_no_shadow** — with no shadowing duplicate, the equation’s head is what is selected (the positive counterpart).
- **h1_head_only_vs_full** — **the theorem that corrects the spec**: H1 read as “for every label m ” holds only in the **rigid-tail** regime; with a flex tail, the substitution-based search recurses into the tail while head-only discharge stops at the head. The weak form is what the implementation needs (and what we claim); the strong form is false as originally written in the companion spec.
- **h1_discharge_head_agrees** — the discharge’s head-reading agrees with the full search on the equation’s own label.

The corresponding probes exist and are pinned: `rowgadt_dup_rebuild` (a full-duplicate rebuild is *accepted* — the rebuild arm handles duplicate equation bodies) and `rowgadt_dup_fewer`, which pins the *boundary* of this story rather than H1 itself: it is a rebuild with *fewer* duplicate occurrences, and it is rejected — the known false reject of the H2-b domain rule, discussed under the escape coarseness below.

4.3 H2-a — store truncation

Truncation is exact and nesting-safe **by construction** — restore assigns the snapshot prefix — and the construction is mechanized as three theorems (`h2a_truncation`, `h2a_no_survival`, `h2a_snapshot_is_suffix`, plus `h2a_inner_snapshot_suffix` for the nesting shape:

an inner branch’s snapshot is a suffix of the outer store, so restore to a prefix is well-defined at any depth).

4.4 H2-b — escape as a domain rule, and the two counterexamples

H2-b is the obligation the adversarial hunt shaped (Section 6.2 states the hunt; here is what it produced): *no equation on a variable free in the branch’s result type may be needed to type that result* — including under an alias, which is where the real hole was. The mechanized results:

- **The domain rule.** `h2b_proper_subdomain` (a refined tail is a proper sub-row of its head), `h2b_domain_rejects_tail` (the drop is rejected), `h2b_domain_accepts_rebuild` (the legitimate rebuild is accepted), `h2b_domain_shrink`.
- **Exactness, proved.** `h2b_domain_exactness`: the rule is **exact for fresh labels and coarse for duplicates** — the boundary is a theorem, not a hedge. The over-approximation is itself a theorem: `h2b_overapproximation` proves the *old* syntactic predicate fires on a legitimate full-row rebuild — i.e. the old check’s false reject was exactly the shape the domain rule fixes (`rowgadt_shape_rebuild` now compiles clean).
- **Both counterexamples, mechanized.** The let-launders hole is closed by `h2b_let_severs_occurs` + `h2b_rigid_preserves_occurs` and **H2b_no_quantify_refined_tail** (a refined tail is never quantified, so its occurrence survives generalization — the generalization predicate `GeneralizesRespectsRefined` carries the exclusion, and `letR_excludes_refined_tails` / `letR_excludes_refined_body_tails` consume R-LET’s actual premises). The wildcard sibling leak is **H2b_wildcard_leak_shape** + `h2b_direct_collapses` (the *symmetric* orientation the old check never tested) + `h2b_wildcard_collapses`. `h2b_alias_identifies` covers the through-an-intermediate-variable aliasing.
- **The residue is coarseness, not unsoundness.** A rebuild with *fewer* duplicate occurrences than the equation body is a false reject (`rowgadt_dup_fewer` errs cannot unify `a` with `{x:Int | a}` — the pinned do-not-weaken fixture). An adversarial battery over duplicate shapes (bare tail return, let-launders tail, wildcard sibling, wrong label, wrong first-occurrence type, wrong field type; re-homed in `examples/research/dup-hunt/`) found **no unsound accept**. The asymmetry is the useful part: H2 errs safe, and the battery bounds what it is missing.
- **Result-scoping, measured.** The tail-aliasing check is *result-scoped*, and so is the obligation: a branch may perform the same flex-tail-aliases-rigid-head identification in its body — `select`’s `There` branch does, recursing on the tail — provided the result type does not mention the row (`rowgadt_select` stays clean). H2-b as stated is exactly result-scoped; strengthening it to “no aliasing anywhere in the body” is refuted by that fixture.

4.5 The update theorem and its dual

The positive row result is mechanized (`Update.lean`, reusing `RowGadt`’s row algebra): `update_domain_preserving` (update preserves the row’s domain: restrict-then-re-prepend of the existing first occurrence), `update_reflexive_domain`, **`update_accepted_at_rho`** (under $\rho \doteq \{ \ell : \tau \mid \rho' \}$, update is well-typed at the abstract ρ with no discharge and no binding), and — the dual — `insertion_fresh_extends`, `insertion_domain_exactness`, **`insertion_rejected_under_refinement`** (a fresh-label insertion strictly extends the domain, which is why it may never consult the store), `insertion_shadow_domain_unchanged`. The restriction side of R-UPD-INS is itself mechanized in the typing arc: `refinedTailVar_iff_insertionRejected` ties the rejection predicate to the store’s

refined-tail lookup, and `insertionRejected_mono_left/right` prove the monotonicity the rule relies on.

4.6 Preservation of the record operations

The deadline-grade slice of T2 is mechanized in `Preserve.lean`: a minimal small-step relation with exactly the two record reductions (selection by first-occurrence lookup; update by cons-prepend-shadow — the *measured* dynamic semantics: records lower to association lists and `assoc` is first-match-from-head, the same first-occurrence discipline the rewrite uses), plus `record_first_occ / record_domain` (the static/dynamic first-occurrence correspondence). `sel_preserves`, `selDisch_preserves` (the *discharged* selection — the refinement-licensed one — preserves types), `upd_preserves`, and the step-level forms (`sel_step_preserves`, `selDisch_step_preserves`, `upd_step_preserves`), with `firstType_subst_rowvar` as the full-grammar restatement of `h1_no_shadow`. The discharged selection and the update cases need **no** unifier axiom; the plain selection case is conditional on `unifies_field_projection` (that unifying two record types relates the first-occurrence types of a common label) — stated as an axiom parameter alongside `Unifies/Captures`.

4.7 What is argued, and the negative result

No general soundness theorem exists for $\lambda\rho G$. T1 (progress) and T2 (preservation) are argued, not proved: T1 introduces no new redex shape and its genuinely new obligation — every selection licensed by R-SEL-DISCH finds its label at runtime — is *carried by* T2’s `SELECT` clause, which is what `selDisch_preserves` mechanizes; T2’s two row clauses are exactly what H1 and the update theorem carry. What the mechanization *does* prove are typing-judgment theorems — the row algebra, the store discipline (`tier_R_forces_escape` and companions), the escape rule, and record-operation preservation — conditional on the unifier abstracted as a parameter (Section 4.1); it does not attempt the whole calculus. We say this once, plainly, and do not let later sections upgrade “argued” to “proved”.

The negative result (stated, not proved). Let $\lambda\rho G^-$ be the variant in which branch mode is replaced by global mode: a branch’s would-be capture is solved into θ . Then $\lambda\rho G^-$ types programs whose exported scheme lies about row domains. For `select` (Figure 1): the `Here` branch’s equation is $\rho \doteq \{ l : t \mid \rho_1 \}$, the `There` branch’s is $\rho \doteq \{ k : s \mid \rho_2 \}$; sequential clause checking solves the first into θ , and the second either fails spuriously (the *policy* shape — today’s rigid guard’s rejection) or, with the guard removed, silently overwrites — and the exported scheme $\forall \rho \ l \ t. \text{Has } l \ t \ \rho \rightarrow \{ \rho \} \rightarrow t$ keeps promising nothing about l , while the body’s $r.l$ was justified by it. The mechanism is measured, twice, in the implementation’s own history: the pre-discharge classic case bound `a := Int` globally at the definition while call sites kept `$\forall a. \text{Expr } a \rightarrow a$` — silent wrong code (`evalInt (BoolLit True)` type-checked at `Bool` and would crash); and the five unsound accepts of Section 6.2 are the same class of defect — a branch equation that should not survive its branch either escaping it or dying without its occurs check. So: the discipline is **necessary** (this theorem-shaped statement, with its mechanism measured) and **sufficient for every program in the corpus and the designed fixtures** (122 gate checks on the typing surface — the gate total is 152, the other 29 being frontend lambda-lift fixtures and one a hand-crafted-bundle run, neither of which exercises the escape rule; the three-pass hunt closed all five unsound accepts, and found none after the final class-level repair) — sufficiency over *all* programs is **not** proved, and is not claimed.

Three-way agreement. The escape boundary is corroborated in three independent places: the Lean theorem (`h2b_domain_exactness`: exact for fresh labels, coarse for duplicates), the implementation’s own exactness note (the rebuild arm accepts exact field-list matches; the fewer-duplicate rebuild is a false reject), and the duplicate-shape adversarial battery (no unsound accept, the one clean accept being the exact full rebuild `e3_swap`). The proof, the implementation, and the probes agree on both the rule and its boundary — that agreement is the paper’s metatheory story.

5 THE PRINCIPALITY BOUNDARY

The discipline has a cost, and the cost is *located* — and the location is counterintuitive.

5.1 The boundary, stated

Principal types are retained in the fragment we call **P**: programs where every refinement’s target is either (a) a branch-scoped existential — the constructor’s own variable, rigid *within* its branch by R-EXISTS — or (b) discharged inside the branch. Plain row code (no GADT) is the degenerate case: `f rec = rec.x` infers $\forall r\ a.\ \{r \mid x : a\} \rightarrow a$, as Leijen’s principality result says it must.

Principality is lost in the fragment we call **Q**: exactly when two sibling branches refine the same *signature* row variable and the result type mentions it. In **Q** there is no principal monotype. The two clauses of `select` infer monotypes that cannot be unified:

```
select Here  r = r.l      -- Has l t { l : t | ρ1 } → { l : t | ρ1 } → t
select There h r = select h r  -- Has l t { k : s | ρ2 } → { k : s | ρ2 } → t
```

for distinct $l \neq k$ there is no single row substitution that solves both equations, so the inferred monotype does not exist — only the *scheme-with-store* reading, which is to say: the signature $\forall \rho\ l\ t.\ \text{Has } l\ t\ \rho \rightarrow \{\rho\} \rightarrow t$ is **checkable, not inferable**. This is precisely the locally-abstract-types dependency: rigidity is what makes the signature checkable under conflicting branch equations, and the signature is not optional in **Q**.

5.2 The measurement, and the inversion

The L3 table (Figure 5): three presentations of the *same access* — read a field justified by a witness — with the signature *omitted*, all three gate-registered and re-measured:

presentation	signature omitted	measured
(i) witness-encoded (Some w x + nested witness match)	inferred — Any → String	rowgadt_l3i: run "3"
(ii) native row + GADT refinement (Figure 1’s select)	rejected — the sibling equations conflict on the flexible ρ (cannot unify {k:a\ b} with {\ a})	rowgadt_l3ii: err ... cannot unify {k:a\ b} with {\ a}
(iii) plain row access (no GADT)	inferred — $\forall r\ a.\ \{r \mid x : a\} \rightarrow a$	rowgadt_l3iii: run 1

Figure 5. The L3 principality measurement — three presentations of the same access, the signature omitted. Source: gate fixtures `rowgadt_l3i`, `rowgadt_l3ii`, `rowgadt_l3iii` (measured column, re-measured by `tools/osier-numbers.sh`; registered rows in `tests/elm-`

fixtures/MATRIX.md); the P/Q classification is §5.1 above (docs/research/row-gadt.md §5).

The middle row is the boundary of Section 5.1, measured. But the informative comparison is (i) vs (ii): the *encoded* presentation sits on the **P side** while its *native translation* sits on the **Q side**. The witness encoding does not merely model rows — **it moves the principality boundary**. The reason is structural: in the encoded presentation the recovered element type is a *constructor existential*, rigid within its branch by R-EXISTS, so inference proceeds without a signature; in the native form the recovered row refines a *signature* variable, which must be rigid for the sibling branches to agree — and a signature-directed rigidity cannot be recovered by inference.

We record the history because it is itself evidence: our pre-measurement guess had the two sides **swapped** (we expected witnesses to lack principal types). The measurement is the result, and the guess is not published as one.

The price of the encoding. The P side is bought at the cost of expressible operations: the encoded presentation has no update (an HList cons-cell rebuild is not a record update), no domain restriction, and the absent-field negative — the `rowgadt_absentfield` rejection that Section 2.4 uses to *bound* what refinement licenses — cannot even be *stated* in it. The boundary is a trade, not a ranking: the encoding buys inference and loses the operation inventory.

5.3 What position-directed rigidity does and does not move

The rigidity work (Section 6.3) was measured against this boundary and does **not** move it: `rowgadt_l3ii`'s `select` is *unsigned*, and a signature-directed rule cannot reach an unsigned definition, so the Q-side rejection stands. What the change did flip is the closely-related bare *HList* form — `hget : Has l t rho → HList rho → t` with no binder, which conflicted at its recursive `There` call before and now checks clean (`rowgadt_hget_bare`). The honest reading: the P/Q line sits where it was — a signature is required exactly where two sibling branches need different equations on a signature variable whose result mentions it — and the narrow delta (bare `hget` checkable, rigidity systematic rather than per-branch) is the *structural* value of the change: it closes the should-be-rigid-treated-as-solvable bug class at its root instead of per case.

6 IMPLEMENTATION AND MEASURED CORRESPONDENCE

The discipline is not a paper design with a sketch: it runs inside a real checker. The substrate is an Elm-family compiler with Hindley–Milner inference over Leijen scoped-label rows (the pre-existing `elm-compiler/src/Type/*` tree: unification with a row-rewrite, let generalization, an eight-library corpus and a self-host track). The change is contained to the pattern path and the unifier state; lowering, codegen, and the VM are untouched (types are erased). Everything below is *measured*, and every number is reproducible from the artifact by one command (`tools/osier-numbers.sh`, which rebuilds the compiler, runs the gate, checks the corpus byte-identity, runs the unit suite, builds the Lean project, and re-derives the interpreter recount; its output is the source of every number printed here — run on the current tree of the artifact repository. The artifact tag `osier-paper-artifact-1` (annotated; it dereferences to the commit it was made on) predates the final ten commits, which touch the documentation, the gate and reproducibility scripts, and the generated fixture registry this paper cites — but not the compiler, the Lean project, or the fixture programs, which are identical between the tag and the current tree, so no number printed here depends on the tag. The implementation was split out of the host project that previously contained it: the import brought the tree across from the host's then-HEAD, eight commits ahead of the freeze tag marking the pre-adversarial-pass baseline, with two further host-side

commits following; the language was later removed from the host and the UI parked. The arrow-spelling GADT parser fix, the equation-lifecycle repair, and the lambda-lifting pass all post-date the freeze tag).

A substrate fact, recorded as an artifact datum. The substrate’s local `let` is *sequential-only*: a local function cannot see its own name, so a self-recursive local helper fails **type error ... unknown name: f** (raised by the checker, before lowering). The compiler’s own `Type/Exhaustive.elm` (its `resolveScrutinee`) hit this while the compiler’s sources were being made self-hosting; a frontend lambda-lifting pass now hoists recursive local groups to the top level (`elm-compiler/src/Frontend/Lift.elm`), with the VM substrate untouched — no new instruction and no collector change, which is why a frontend pass was chosen over a VM-level `letrec`. It is recorded as a measured datum about what the substrate is and what the artifact needed, not as a contribution.

The non-regression invariant. The corpus — the compiler’s own eight core libraries, the prelude, and the runtime, as compiled artifacts — must be **byte-identical** before and after the change: the 149 compiled artifacts match a committed sha256 manifest entry-for-entry with zero moved (re-measured, exit 0). This invariant is what makes the discipline’s addition safe to *adopt*, and the design pays for it deliberately: the historical inference path remains primary; the new machinery (branch-local capture, discharge, the declaration-directed retry) engages only where the historical path fails — so every pre-existing diagnostic is unchanged by construction. The regression gate passes 152/152 (the designed-fixture matrix with expected outcomes is Table 1 in Appendix A, the complete generated registry is `tests/elm-fixtures/MATRIX.md`, and the gate is `tests/elm-fixtures/run-elm-gate.sh`), and the unit suite passes 114/114.

6.1 What the build surfaced: kinds

The first four implementation blockers the build hit were all *pre-existing substrate defects*, reproducible without any GADT, and **three of the four were kinding defects**:

1. ADT generics were forced to **KType**, so a generic used at a row-tail position mis-kinded (`Has l t ρ` with ρ used as a record tail). Fixed by kinding generics from their use position.
2. Kind inference by first *use* position then broke the other source order — the same declaration mis-kinded depending on whether the tail use preceded the index use. Fixed with a row-tail **prescan** that binds record-tail names to **KRow** before conversion (`collectFile TypeKinds`).
3. The unifier had no path for a bare **KRow** variable against a record type — exactly the shape a GADT constructor’s result index produces. Routed to the row binder in **branch mode only** (the global path keeps its historical behaviour; the bare-**KRow**-vs-record gap is still measureable outside branch mode — see the `read` reducer in Section 1.4).
4. A flex-alias wrapper bug: the flex row variable was bound to the record *wrapper* rather than to the variable, so a rigid variable meeting its own alias was rejected. Fixed in the identity case.

The finding generalizes: **a calculus without explicit row kinds cannot predict where a real implementation breaks**. The declarative spec states the kind of every declaration; the implementation *elaborates* it from the surface (because the surface has no kind annotations), and the elaboration rules were built precisely because first-use inference mis-kinded order-dependent programs. This is why λ pG carries kinds in its syntax (Section 2) rather than as a periphery.

6.2 The escape check’s evolution — the section’s centrepiece

The branch-result escape rule (Section 3.1’s Tier-R) went through three measured stages, and the stages are the paper’s honesty about its own metatheory.

(1) **The first check was syntactic** — a tail-occurrence test in one orientation: reject when the refined row’s *tail* variable appears in the body’s type while the *head* appears in the expected result. It was sound on the shapes its author considered.

(2) **The first of three adversarial passes found two programs the check accepted that are unsound** (both are now gate-registered compile errors, `rowgadt_escape_laundry` and `rowgadt_escape_wildcard`; both programs in full, verbatim, are Figure 6):

- **Let-laundrying.** The `Here` branch refines $\rho \doteq \{ l : t \mid \rho' \}$, the scrutinee is an `HList` ρ , and the branch returns its tail — laundered through a `let` (Figure 6a). `let`-generalization *quantified the refined tail* ρ' , so the use of `ys` instantiates a fresh variable with no link to the tail; the occurrence test sees nothing; the fresh variable then legally aliases the rigid head. The returned list is missing field `l` — an unsound accept (the pre-fix checker compiled it clean).
- **Wildcard sibling leak.** The refining branch returns the tail while a wildcard branch returns the full row (Figure 6b). The shared case-result variable lets the wildcard’s full-row result unify *tail* := *head* — a legal flex alias — zonking the tail away before the clause-level check runs. The check had been written in one orientation only; the sibling branch smuggled the identification past it.

(a) **Let-laundrying** — verbatim from `tests/elm-fixtures/rowgadt_escape_laundry.elm` (lines 11–31):

```
type Has l t rho
  = Here : Has l t { l : t | rho }
  | There : Has l t rho -> Has l t { k : s | rho }

type HList rho
  = HNil : HList {}
  | HCons : t -> HList rho -> HList { l : t | rho }
```

```
escapeBad : type l t rho. Has l t rho -> HList rho -> HList rho
escapeBad h xs =
  case ( h, xs ) of
    ( Here, HCons _ rest ) ->
      let
        ys = rest
      in
        ys

    ( There h2, HCons _ rest ) ->
      escapeBad h2 rest
```

(b) **Wildcard sibling leak** — verbatim from `tests/elm-fixtures/rowgadt_escape_wildcard.elm` (lines 11–27):

```
type Has l t rho
```

```

= Here : Has l t { l : t | rho }

type HList rho
  = HNil : HList {}
  | HCons : t -> HList rho -> HList { l : t | rho }

escapeBad : type l t rho. Has l t rho -> HList rho -> HList rho
escapeBad h xs =
  case ( h, xs ) of
    ( Here, HCons _ rest ) ->
      rest
    _ ->
      xs

```

Figure 6. The two stage-(2) counterexamples. Pre-fix, both compiled clean (unsound accepts); post-fix, both are gate-registered compile errors, `err escaping row equation ...` (Table 1).

(3) **The rule is now domain-based**, and the domain rule is what the mechanization proves sound. At a branch result whose unification fails on a refined rigid row head, the check decides on the *domain*, not on syntactic occurrence: **drop** (the unification newly identified the tail with the head — **dropIntroduced** compares the tail-reach before and after this step) → reject; **rebuild** (the presented row unifies with the equation’s body, and the head does not occur in it) → *accept*; anything else → reject (code seats: `tailReachesHead`, `dropIntroduced`, `rebuildMatches` in `Type/Infer.elm`). The rebuild arm is what makes the rule *complete enough to be usable*: returning the scrutinee rebuilt — `HCons x rest at HList { l : t | ρ' }`, whose domain equals the head’s under the equation — is a *legitimate* return that the syntactic check false-rejected (`rowgadt_shape_rebuild`, `clean`). Both fixes and the rebuild acceptance are pinned; six further must-error escape shapes were re-verified after the change.

The two passes after the escape check. The escape rule was only the *first* adversarial target. A second pass attacked the checks *as they ship* — the previous pass had only ever attacked the rule it replaced — and found two more unsound accepts. **CE-1**, a **dropIntroduced** scoping bypass: the body pre-aliased the refined tail to its head *before* the result unify, outside the window the check observed (`rowgadt_ce1_prealias`). **CE-2**, a bare-variable scrutinee over-generalisation that also hit *plain ADTs*: matching a bare scrutinee lifted and then dropped its index equation, over-generalising to $\forall a. a \rightarrow r$ and over-refuting the sibling, so `f B` compiled clean and crashed at runtime (`rowgadt_ce2_barevar`; the same defect in a non-GADT form, `rowgadt_ce2_plain`). A third pass attacked the *coercion surface* — the Tier-T result-unify path the whole two-tier rule rests on, which had been assumed correct rather than tested — and found the most serious accept so far: a **wrong-typed value**, not a crash. `MkBox : a -> Box (List a)` with `get b = case b of MkBox v -> v` accepted `get (MkBox 3) at List Int` while its value is the `Int 3`, because the *flex*→*rigid* alias bound the equation $a \sim \text{List } a$ without ever occurs-checking it (`rowgadt_ce3_fieldalias`). A per-shape guard (`refinedTargetAliasedBy`) closed that one shape, but a review found **three bypasses** of it — the alias forming inside a tuple, before the result unify, and through a `let` — each compiling clean and printing a wrong-typed value. The real cause was the **equation lifecycle**: a pending `KType` equation must be

occurs-checked before it is discarded or a result accepted; the repair (`pendingTypeOccurs`) does exactly that, and all three bypasses now error `infinite type(rowgadt_ce3a_tuple, rowgadt_ce3b_prealias, rowgadt_ce3c_let)`. In total: three passes, five unsound accepts, one class-level repair.

The stage-(2) lesson is methodological and worth stating: the hunt was run *before* the mechanization, and H2 as first conceived was the wrong shape of lemma — a statement constraining only the branch-result unification cannot see the let-form leak. Had the theorem been proved first, it would have enshrined a property the implementation did not enforce. Section 4 gives the mechanized form of both counterexamples (`H2b_no_quantify_refined_tail`, `H2b_wildcard_leak_shape`) as part of the metatheory, not as an appendix.

6.3 The type **a**. surface and the too-general police

The surface prefix `type  $\hat{x}_1 \dots \hat{x}_n$` names the signature quantifiers that are **rigid while the body is checked**; unnamed quantifiers stay flexible, and soundness is restored *after* the body by the **too-general police** — a re-check that skolemizes the declared scheme and unifies it against the generalized body type, rejecting “annotation is too general”. The measured asymmetry is exactly the design’s prediction: the binder’d `select` checks; the same body without the binder is rejected — either by the conflicting-equations failure (`cannot unify {k:a| b}` with `{| a}`, `rowgadt_l3ii`) or, for classic GADTs, by the police (“annotation is too general”). Rigidity is the mechanism; the prefix is its surface.

Position-directed rigidity. A signature variable at a *direct GADT-index position* (`Has l t  $\rho$` , `Expr a`) is rigid whether or not the surface binds it: the checker collects the file’s own annotated-result constructors and their index variables (`collectGadtNames / gadtIndexVars`) into the scheme’s bound set, so `hget : Has l t  $\rho$  -> HList  $\rho$  -> t` — with **no** binder — now checks identically to its binder’d form (this specific program flipped from rejected to clean and is pinned: `rowgadt_hget_bare`). The measurement was honest about its own narrowness: bare `select/eval` *already* compiled clean (the per-branch index lift covered them), and the principality boundary of Section 5 does **not** move (`rowgadt_l3ii` is an *unsigned* definition, which a signature-directed rule cannot reach). The value of the change is structural — it closes the recurring “should-be-rigid variable treated as solvable” bug class at its root, and it is what makes the refutation guard of Section 1.4 sound at a *bare* index (`refutbare`).

6.4 The compiler’s own effect interpreter

An artifact datum, recorded as such. The `Task` type now declares only the effects its host can perform — the ten UI effect constructors and the `Key/Mouse` types left the language and park in the host project pending a UI redesign — and an effect the host does not implement fails loudly and terminates, pinned by a hand-crafted-bundle gate check (`unhandledtask`). It is recorded as a datum about what the artifact now is, not as a contribution.

The acid test is not a fixture but the compiler’s own `Runtime.runTask` — the free-monad interpreter for its 20-constructor effect type, historically a *trusted body* (a function the checker skips, with a lie for a type). The trusted comment stated its own retirement precondition: kept trusted “until the per-ctor result table can drive a checked interpreter”. The machinery of this paper is that table’s typability condition, so we tested it.

The precondition was met, mechanically. The 20 per-ctor result annotations (`TaskExec a : Task x (Int, String, String)` — real source syntax in `Runtime.elm`, migrated from a hardcoded table) were proven behaviour-preserving by *two independent oracles*: the inferred constructor schemes were byte-identical before/after the migration (20/20), and the full corpus compiled byte-identically.

And it was not sufficient — a fail-fast artifact. Un-trusting `runTask` (removing it from the trusted list) reports exactly *one* error, at `TaskExec`, and stops: a checker that halts at the first error cannot see the branches behind it. Masking each failing branch in source order to reveal the next (a reproducible bisection script, `tools/osier-recount-runTask.sh`, which edits only a scratch copy and asserts every step’s expected error) gives **four** failing branches — so the honest count is **16 of 20 branches check honestly**, and the earlier “29 of 30” figure was this fail-fast artifact (in the 30-constructor era, before the UI effects left the `Task` type). The four, named:

branch	failure	class
<code>TaskExec</code>	payload <code>a</code> vs <code>execPlan Prim : List a → List b</code> needs <code>a ÷ List a</code>	defect: a dynamic cast
<code>TaskNow</code>	<code>Ok 0</code> number literal hits flex-marker conflict before discharge fires	defect: literal handling
<code>TaskQuit</code>	un-annotated nullary; body <code>Ok ()</code> vs abstract index <code>a</code>	design: deliberate generalization
<code>TaskStat</code>	result is a <i>closed record</i> ; the Tier-R over-approximation refuses any record body	defect (known): Section 4’s coarseness

One of the four is *design, not defect*: `TaskQuit` is the one constructor the annotation table deliberately left generalized, and the bisection’s own terminal signal proves the list is closed — after masking the fourth failure the trivial fixture compiles **clean**, the no-fifth-branch evidence. The composition of the holdouts moved, and the paper says so rather than reading the new figure as a code change: `TaskGuiPoll`, the second deliberate holdout of the 30-constructor era, left the `Task` type with the UI effects — and with it went the old terminal signal (a `Tea.GuiEv` call-site cascade at a `Task.perform` use), which was the earlier evidence that annotating the generalized constructors would break their callers’ deliberate polymorphism. The count has one stated condition: it is taken under a `type x a.` binder (the committed signature has none; without the binder the flexible index is bound by the first concrete-result branch and the later branches cascade — the discharge being measured requires the rigid index). The condition is printed by the recount script, not hidden.

The residue is a representation problem, not a refinement one. `TaskExec`’s payload is typed `a` — the constructor’s existential, absent from its result — while the runtime value *is* a tagged list; no refinement of an abstract variable expresses `a ÷ List a`. The honest fix is a surface change (type the payload `List a`), not more type theory. The rest of the trusted list decomposes the same way: the remaining trusted bodies are lies of *representation* (the VM value is uniformly typed; `compare` introspects it), not of refinement. An experiment with an honest `Value` datatype (`probeA/probeA2` in `examples/research/`) confirms it: with honest constructors, the `compare/decode*` bodies type-check clean, and a deliberately wrong body is still rejected — the checker’s discipline is intact; what is missing is an honest datatype, not an equation.

6.5 The measured correspondence to the calculus

Where the implementation approximates the declarative rules (the calculus’s own `[U]/[S]` inventory, compressed):

- **The two-pass retry** — [S], **soundness-neutral**. The implementation checks a signa-tured declaration’s clauses bottom-up, and only on failure with a directly-case body retries declaration-directed (each branch against the declared result, each with its own discharges). The *declarative calculus has only the declaration-directed rule*. The retry accepts a subset: the canonical evaluator wrapped in a `let` is spuriously rejected (`gNeg_letcase`, re-homed and re-measured: `cannot unify Bool with Int`). No completeness claim is made for the retry path.
- **Per-file kinding** — [U, weakly]. ADT generic kinds are collected per file, so a cross-file bare row parameter could kind as a type variable (turning a row equation into a type equation Tier-T would discharge). Unexercised by the corpus and fixtures; stated as an approximation.
- **The duplicate-label coarseness** — [S], **proved and pinned**. The rebuild arm requires an *exact* unification with the equation body, so a body rebuilding with fewer duplicate occurrences is a false reject (`rowgadt_dup_fewer`, with a do-not-weaken comment); the domain rule’s exactness boundary is a theorem (`h2b_domain_exactness`, Section 4).
- **Trusted bodies and surface restrictions** — [S]. The fixed trusted list is modelled as an escape, not a rule; the surface has no type-level strings; record literals are closed (an open row can only be *named*, never constructed); `{ | ρ }` bare-open does not parse; the update base must be a local variable.
- **Exhaustiveness and refutation** — [S], **an addition**. The calculus’s case rule now carries a compile-time exhaustiveness and refutation check (`Type/Exhaustive.elm`): a miss-ing possible arm is an error; an impossible arm under the branch equations is refuted and not required. The check’s own boundary is measured and pinned: refutation fires only at a concrete or binder-rigid index (`refutbare`); a *pre-existing* false positive for unsigned nested patterns (`Just (a, b)` vs `Just _`) remains, and is not a soundness issue.

6.6 Runnability: the gate now executes, not just compiles

The `compile_clean` gate check asserts only that compilation *succeeds* — it never runs the result — so a lowering or arity bug in the GADT path was invisible for the whole project, until one was found: a GADT constructor in the arrow spelling `C : A -> B -> T` parsed with `arguments = []`, so the lowerer emitted an arity-0 constructor and *applying* it crashed (`apply non-callable`), while the type checker was happy because the result annotation is a function type. Seven GADT fixtures were converted from `compile_clean` to real run checks with pinned outputs — `rowgadt_eval [cons 1 . true]`, `rowgadt_het "hi3"`, `rowgadt_l3i "3"`, `rowgadt_l3iii 1`, `rowgadt_fsm "open:42"`, `rowgadt_fsm_nested "open:42"`, `refutpos 5` — so the gate now *executes* the GADT path, and this arity class is a regression fixture rather than an invisible gap.

6.7 Provenance: AI assistance in building this artifact

The ACM Policy on Authorship requires AI use that *conducts the research* — designing the methodology, implementing, testing, validating, and archiving the artifacts the conclusions rest on — to be described in detail in the methods section of the work; AI assistance with the *writing* no longer requires disclosure. This subsection is that description (§6 is this paper’s methods section). It is a statement about process, not about the discipline.

Where the assistance was. AI coding agents, working from the author’s briefs and reviewed by the author, produced the project’s own code, by area: the compiler implementation — the Elm-family compiler and its Hindley–Milner checker over scoped-label rows (`elm-compiler/src/`

, including the eight core libraries), the frontend lambda-lifting pass (`elm-compiler/src/Frontend/Lift.elm`), and the host effect loop (`src/effectloop.zig`); the research extension itself — the branch-local refinement machinery in `Type/Unify.elm` and `Type/Infer.elm`, with its surface (the `type a.` binders, exhaustiveness and refutation checking); the Lean mechanization (`lean/`); the verification harness — `tools/osier-numbers.sh`, the fixture gate (`tests/elm-fixtures/run-elm-gate.sh`), the corpus baseline (`tools/osier-corpus-baseline.sha256`), and the recount script (`tools/osier-recount-runTask.sh`); the adversarial test passes — the three-pass unsound-accept hunt and the fixtures pinning its findings; and the artifact packaging — `ARTIFACT.md`, the generated fixture matrix (`tests/elm-fixtures/MATRIX.md`), and the prerequisite preflights. The compiler substrate is *pre-existing* in §6’s sense — it predates the contribution — but it was built inside this project’s sessions, with the same assistance.

The degree, as the record shows it. Every commit in this repository’s history carries the author’s name (`git log --format='%an' | sort | uniq -c` prints the author’s name and nothing else). The commit record alone therefore says nothing about the assistance: it credits the author for work that the project’s working records show was written by AI coding agents — several underlying models, dispatched per work unit — with the author directing and committing the results, and two early commits in the parent repository from which this one was split carry the coding agents’ own sandbox identity. The record does not support a per-line or per-component percentage. Parts of the artifact are *ports of third-party code*, not original code by anyone — the vendored elm-syntax parser (from the Elm compiler), the zinc-vm (a Zig port of the Shen ZINC VM), and the elm/core core libraries; there, the assistance was in the porting and adaptation.

The human role. The author directed the work, made the design decisions (the discipline itself, the naming, the scope, and what not to claim), constructed the motivating example, and independently re-verified the headline numbers before they were printed here. No AI system is an author: under the ACM policy, generative AI tools cannot be listed as authors under any conditions, and the author is accountable for the content regardless of its source. The deposit-side note states the same facts for a reader who arrives at the code first (`ARTIFACT.md §8`).

For completeness, the non-required half: drafting and review of this paper’s prose were also AI-assisted. The policy does not require that to be disclosed; it is stated here because the boundary between the two halves is this project’s actual history. —

7 RELATED WORK, ORGANIZED BY THE FEATURE-PAIR MAP

The organization of this section is itself a contribution. Three features — **R** row polymorphism, **G** GADT refinement, **A** explicit/locally abstract polymorphism — pick out cells by the pairs they combine, and the map (26 rows — 24 retrieved from primary sources, 2 carried second-hand; the survey with verbatim quotes and URLs is the companion `docs/research/osier-related-work.md`) makes two things visible at once: *which adjacent cells are occupied* — so that “but X did rows” and “but Y did GADTs” are answered before they are asked — and *that one specific cell is empty*. No published type system refines a row variable by a branch-local GADT equation, and no published system therefore says which record operations remain typable when such an equation is active. (The nearest sounding sentence in the literature — CORELINKS’ “our case construct refines variant types” — is examined in §7.2: presence flip, variant row, no equation.) The claim is about published systems with a *designed discipline*; it is never worded “rows and GADTs have never been combined” — OCaml ships the raw combination as an idiom, and its boundary is discovered empirically by its users.

The claim is falsifiable, and the survey that backs it states the method. Every row of the map is a published system — 24 of the 26 retrieved from a primary source (paper, language documentation,

or maintainer record), 2 carried second-hand — and classified by the feature pairs it ships; the survey records the query terms run and a would-be-breakers table giving each candidate examined and why it falls short of the cell. A referee who knows a published system with a *designed discipline* that refines a row variable by a branch-local GADT equation is asked to name it: one such system occupies the cell and refutes the claim. The rows-only side of the map carries the method’s clearest positive control: PureScript ships row-polymorphic records and its core team has argued GADTs out — “I think we should commit to not adding GADTs” (hdgarrood, 2019) — so the pair cannot arise there by first-party decision rather than by oversight.

7.1 The rows-only lineage (R)

Wand 1989 types record *concatenation* and loses principality for it — “this calculus does not have principal types, but does have finite complete sets of types” — compensating with finite complete sets. Our fragment Q loses principality for a different trigger (refinement-with-mentioning-result, not concatenation): **the same casualty, a different mechanism — we do not conflate the two**, and Wand’s compensation suggests a question for future work (does Q admit finite complete sets?). **Rémy 1994** is the reference point for the operation dial: unrestricted extension unconditionally (**new_a**), presence flags as quantification (System II*), and update-as-extension — the expressiveness maximum our conditional insertion rejection trades against (Section 3.3). **Gaster & Jones 1996** dial extension to strict under a lacks predicate and type update as restrict-then-extend — the second published position on the dial. **Ohori 1995** replaces row variables with *kind restrictions* on instantiation — worth citing against our kinding findings (Section 6.1): the kinds are load-bearing, but without a row variable there is nothing to refine — the seat itself is absent. **Leijen 2005** is the substrate: scoped labels, the eq-swap rewrite, and the duplicate-label semantics our H1 commutation is stated over; the paper predates GADT inference and contains no occurrence of refinement. **Paszke & Xie 2023, Spanò 2024, Hubers et al. 2025** extend the R lineage (infix extension for tabular data, overloading conversion, row-typed recursion) with zero GADT content — grep-measured on the retrieved texts. **Toohy, Chen, Jamalzadeh & Xie 2026** occupies the adjacent **rows × type-classes** cell, not the R lineage: dictionary-passing elaboration of ad-hoc polymorphism over polymorphic rows, with **All** (a property across all fields), **ind** (a fold over rows), **Split/Lift**, and commutativity annotations over a *parameterised* row theory, all mechanized in Lean 4 — and the grep on its 38-page full text is **GADT 0, refine 0, refinement 0, branch 0**: no equations on rows, so its delta from us is exactly the refinement interaction. That cell is **out of scope** here: we claim nothing about rows × type classes, and generic structural dispatch over rows is their result (**All + ind**), not a consequence of our discipline.

7.2 Presence: the nearest neighbour in one dimension

CORELINKS [Lindley & Cheney 2012] is the closest published system to answering “what must an operation know about which fields are present?”: it types insert/update/delete against (**label × presence × type**) rows by *quantifying* a presence variable over the needed row — “presence ... is unconstrained as we can infer nothing about it solely from an insert or update operation”. This is **abstraction** — the opposite direction from our *assumption* on a rigid variable inside a branch — and it has no equations, no rigidity, and no branch-local store. Its rows also have **distinct labels**, which is precisely the case where our domain rule is *exact* (Section 4.4): the duplicate-label coarseness is a genuine expressiveness difference between the systems, in both directions. One sentence in CORELINKS sounds close to our cell: “our case construct refines variant types, whereas OCaml’s equivalent does not” (its §2.1). What that case construct does is flip a **presence flag** — the CASE rule refines “the type of the value being matched” so that “the non-matched label is absent” in the type of the variable bound by the default branch — and it does so on a **variant**

row, adopting no equation: the words **GADT**, **rigid**, **skolem**, and **equation** never occur in the paper (grep-measured). Our cell is none of these: a branch-local equation $\rho \doteq \{ \ell : t \mid \rho' \}$ on a record’s row variable. **Castagna & Peyrot 2025** give presence polymorphism over optional fields in a set-theoretic system and bracket refinement explicitly: “type cases, guards, or type narrowing ... seem mostly orthogonal to the introduction of row polymorphism” — the live assumption our cell falsifies for the product half.

7.3 The GADT-inference lineage (G)

OutsideIn(X) [Vytiniotis et al. 2011] is the standard baseline: implication constraints with *touchables* — variables the solver may unify inside a given constraint. It cannot state our problem: its type grammar $\tau ::= tv \mid \text{Int} \mid \text{Bool} \mid [\tau] \mid T \tau$ has no record type, no row variable, no presence (grep-measured). **Stratified inference** [Pottier & Régis-Gianas 2006] introduces type equations at case constructs and propagates them through annotations — the annotation-meaning caveat we share (“moving a type annotation into or out of a case construct can change its meaning”), with rows never appearing as types. **Wobbly types** [Peyton Jones et al. 2006] put rigidity in the environment per annotation; the word “row” does not occur in the paper. **Ambivalent types** [Garrigue & Rémy 2013] formalize OCaml’s locally abstract types — a rigid variable “can be refined by GADT pattern matching” — with scoped equations that may not leak out of a branch: the closest *mechanism* to ours, on a grammar $(\tau ::= \alpha \mid a \mid \tau \rightarrow \tau \mid \text{eq}(\tau, \tau) \mid \text{int})$ that cannot state $\rho \doteq \{ \ell : t \mid \rho' \}$. **Choice types** [Chen & Erwig 2016] are the nearest G-lineage neighbour to our boundary: they carry branch refinements *in the type language* — choice types $D(\varphi)$, alternatives synchronised by name, with a separate reconciliation phase that replaces choices by type index variables — and price it in their own words, “principality comes at the price of having choice types in the type language”. Their grammar $(\tau ::= \alpha \mid \tau \rightarrow \tau \mid T \tau)$ has no record type, no row variable, and no presence (grep-measured on the retrieved text): refinement reaches only GADT type-constructor parameters, never a row — they stop at the boundary this paper crosses.

HMG(X) [Simonet & Pottier 2007] is the **un-walked door**. It is a parameterised, constraint-based account of guarded ADTs — every branch checked “under different assumptions about the type variables in scope” — that could in principle host a row theory: X is a parameter. But no published instantiation of it (or of OutsideIn’s X) targets a row/record theory — the only Rémy citation in HMG(X) is the *object* calculus — and the two row-specific obligations our discipline needs, $H1$ ’s commutation under scoped-label **duplicates** and the **domain** escape rule, do not come out of the generic frame for free. $\lambda\rho G$ is that instantiation, with the two obligations carried explicitly (Section 4). This is the strongest positioning sentence available: we claim the instantiation and the two row obligations, not a new frame.

7.4 The combination cell and its occupants

OCaml is the one widely used system with all three features, and its record is the paper’s strongest single piece of prior-art evidence: restricted in the first implementation (Garrigue & Rémy 2012, verbatim, on the same row-variable invariant our discipline protects), still “not well specified” per its maintainers (octachron 2024), with poly-variant patterns blocking refinement outright (“GADT equations cannot narrow a polymorphic variant constraint” [octachron 2024]; the earlier, stronger 4.00-era restriction — “types cannot be refined if a pattern-matching contains polymorphic variant” — was lifted in 4.01 [Garrigue, #5724 rev 13221; 4.01 Changes]), object/GADT invariance (“GADTs are incompatible with object subtyping: they are always invariant with respect to their type parameters”), and the recommended workaround — one row variable per field — being a manual membership-witness encoding, the idiom practitioners already reach for (the discuss.ocaml.org

FSM/reducer threads are the user-demand datapoints, and Section 1.4’s program is one of them, translated). **Flix** restrictable variants [Madsen et al. 2023] refine a *label-set* index under **choose** while the record row index is only ever extended — refinement never flows into the row — and name GADTs as “interesting future work”. **GHC’s HasField** on record GADTs is class-triggered unification over *nominal* records — no row variables. **Koka** uses rows for effects with duplicate labels inherited from Leijen, and its eliminations *unify* the effect row (**catch** unifies μ with $(\text{exn} \mid \mu')$) — never refine it; we cite it for the effect half of the duality and claim nothing about rows-and-effects.

7.5 The frontier

Omnidirectional inference [O’Brien, Rémy & Scherer 2026] restores principality for fragile features (record-label overloading; semi-explicit polymorphism) by reordering constraint solving — and names GADTs as the application they “would be interested in studying”, over *nominal* records. The field is actively rebuilding inference around the principality casualty our boundary measures; our result lands the row-side answer next to it. **Fan, Xu & Xie 2025** implement touchability as *levels* in production checkers — useful vocabulary for the implementability of R (our rigid set) — with GADTs explicitly deferred.

8 LIMITATIONS

We state the limitations as first-class results, not as apology: several of them are theorem- or fixture-backed boundaries (marked), and none is silently hedged.

- **No general soundness theorem** (*paper-gap: the wording is the fix*). Progress and preservation are argued, not proved. The mechanization’s 90 theorems — over 3 axiom parameters — are typing-judgment theorems: the row algebra, the store discipline, the escape rule, the update theorem, and record-operation preservation, conditional on the unifier abstracted as a parameter. What they carry are exactly the two row-specific obligations the argued soundness rests on; the whole judgment is not covered. Later revisions must not upgrade “argued” to “proved”.
- **The escape check is sound but over-approximate — proved, not hedged** (**h2b_overapproximation**): it rejects some legitimate returns. The duplicate-label case is *coarse* — proved (**h2b_domain_exactness**: exact for fresh labels, coarse for duplicates) — and its false reject is pinned by a do-not-weaken fixture (**rowgadt_dup_fewer**). “Fixing” it by weakening the check would reintroduce the unsoundness class the hunt closed.
- **Fragment Q is checkable, not inferable**. Signatureless native row-GADT code with a result-mentioning refinement is rejected; the signature is not optional there (Section 5). The retry path is additionally incomplete: a GADT case nested under a **let** is spuriously rejected (**gNeg_letcase**) — soundness-neutral, and no completeness claim is made.
- **Refutation is conditional**. Exhaustiveness and refutation checking exists and is pinned (**adtgaps**, **refutpos**, **refutneg**, **refutbare**), but refutation fires only where the scrutinee’s index is **concrete or binder-rigid**; a bare flexible index never licenses refuting a sibling (that condition is itself pinned by **refutbare**, after the check initially accepted a partial function through a flexible index — found and fixed during the work). GADT-ness is classified per file, so a hypothetical cross-module GADT index degrades to the old flexible behaviour (safe, via the too-general police and the refutation guard; unexercised by the corpus). One pre-existing false positive for unsigned nested patterns remains.

- **Per-file kinding** (*stated as an approximation*). ADT generic kinds are collected per file, so a cross-file bare row parameter could kind as a type variable — a soundness-relevant approximation, unexercised by the corpus and fixtures. The declarative spec states kinds; the elaboration is a front-end concern.
- **Rank-2 / higher-rank types are absent** — the type representation has no quantifier node, so the general handler signature (a polymorphic continuation) cannot be stated. Deliberate: orthogonal to the row×GADT claim, and landing it would move the evidence baseline under the paper.
- **Surface gaps** (*all completeness, not soundness*): no type-level strings (labels are surface strings; `Has "x" t p` is inexpressible, so the example fixes `l` by the record's signature instead); record literals are closed (an open row can only be *named*, never constructed); `{ | p }` bare-open does not parse; the update base must be a local variable.
- **The membership-witness reducer is definable but not callable at a concrete witness** — read Here `{ n = 1, closed = 2 }` errs cannot unify `a` with `{l:a|b}`, the global bare-KRow-variable-vs-record gap outside branch mode (Section 1.4); the FSM example keeps the witness as a shape, not a working reducer. Relatedly, argument order matters for the FSM reducers (record-first checks; the symmetric event-first form is rejected at its call site) — an inference-order artifact of the implementation, which we report and do not attribute to typing-rule bias.
- **The trusted-body residue is a representation problem** — the interpreter's holdout is a dynamic cast at the VM boundary (`a ≐ List a`), and the remaining trusted bodies decompose into lies of representation, not of refinement (Section 6.4): the honest fix is a surface change or an honest datatype, not more type theory.
- **Evaluation is self-corpus plus designed fixtures** — byte-identity is the *non-regression* evidence, not a benchmark claim, and we claim no user-facing benefit; we claim a characterized boundary. The one external program (Section 1.4) is a translation with three stated boundaries, not a corpus study. The adversarial hunt was three passes: five unsound accepts found and fixed (two in the first escape check, two in the shipping checks, one in the coercion surface, the last closed by a class-level repair), and one runnability bug the gate could not see; absence of more is evidence, not proof. No performance numbers are claimed (the retry fires only on the failure path).

9 CONCLUSION

The conflict between row polymorphism and GADT refinement is an artifact of eager global solving, not of the features: a row variable under a branch-local equation is coherent, useful, and *characterizable* once the equation is captured rather than solved, type equations are discharged at results while row equations never are, and the discipline's costs are located — principality in exactly the fragment that refines a signature variable through two sibling branches, expressiveness in the insertion the discipline must reject. The discipline is implemented, measured against a byte-identical corpus, mechanized where it is load-bearing, and falsifiable where it is not: the fixtures that pin its boundary are the artifact's regression gate. What remains open — general soundness, the refutation condition's generalization, the restrictable-variant dual (the calculus is phrased so the dual statement is a polarity flip), and whether fragment *Q* admits Wand-style finite complete sets — is stated, not hidden.

APPENDIX A: THE FIXTURE MATRIX (THE EVIDENCE SPINE)

Every designed program this paper cites, gate-registered and re-measured on the current tree (`tests/elm-fixtures/run-elm-gate.sh`, `PASS=152 FAIL=0`; the gate also holds the pre-existing corpus fixtures). The oracle is the compiler's output artifact, never the process exit code; the `run` rows execute the fixture through the VM and diff its printed value against a pinned expected output (Section 6.6), while `clean/err` rows assert the compile outcome only.

Table 1. The fixture matrix (the evidence spine). Source: the gate `tests/elm-fixtures/run-elm-gate.sh`, whose complete registry is generated as `tests/elm-fixtures/MATRIX.md` (152 rows, by `tools/gen-fixture-matrix.sh`; staleness check `tools/gen-fixture-matrix.sh --check`) — this table is the paper-cited subset, with the claim each row pins; where the two disagree, the registry and the gate win. The `measured` column is re-derived by `tools/osier-numbers.sh`.

fixture	claim pinned	measured
<code>rowgadt_select</code>	the Figure 1 program: capture + discharge types <code>select</code>	<code>clean</code>
<code>rowgadt_setx</code>	update under refinement; domain-preserving, no escape	<code>clean</code>
<code>rowgadt_absentfield</code>	selection licensed only by equation heads	<code>err type variable a is rigid ... {zz:a\ b}</code>
<code>rowgadt_escape</code>	a row equation needed at the result escapes	<code>err escaping row equation ...</code>
<code>rowgadt_eval</code>	Tier-T discharge; the canonical evaluator	<code>run [cons 1 . true]</code>
<code>rowgadt_evalbad</code>	Tier-T re-check rejects a wrong body	<code>err escaping row equation ... (to String)</code>
<code>rowgadt_het</code>	heterogeneous container: rigid existentials + use-discharge	<code>run "hi3"</code>
<code>rowgadt_noescape</code>	a constructor existential may not escape its branch	<code>err type variable a is rigid ... String</code>
<code>rowgadt_l3i</code>	L3 row (i): the witness encoding <i>infers</i>	<code>run "3"</code>
<code>rowgadt_l3ii</code>	L3 row (ii): the native form is <i>not inferable</i>	<code>err cannot unify {k:a\ b} with {\ a}</code>
<code>rowgadt_l3iii</code>	L3 row (iii): plain row access <i>infers</i>	<code>run 1</code>
<code>rowgadt_hget</code>	the witness-encoded HList, recursive case	<code>clean</code>
<code>rowgadt_hget_bare</code>	position-directed rigidity's one genuine flip	<code>clean</code>
<code>rowgadt_hget_escape</code>	H2-b direct: the tail at the head's type	<code>err escaping row equation ...</code>
<code>rowgadt_hget_badhead</code>	wrong body at the recovered index	<code>err type variable a is rigid ... String</code>
<code>rowgadt_escape_launder</code>	the let-laundering counterexample, closed	<code>err escaping row equation ...</code>

fixture	claim pinned	measured
rowgadt_escape_wildcard	the wildcard sibling leak, closed	err escaping row equation ...
rowgadt_ce1_prealias	CE-1: the dropIntroduced scoping bypass (pre-alias before the result unify)	err escaping row equation ...
rowgadt_ce2_barevar	CE-2 (GADT form): a bare scrutinee over-generalises and over-refutes	err cannot unify Int with String
rowgadt_ce2_plain	CE-2 (plain-ADT form, not GADT-specific)	err cannot unify ...Expr with String
rowgadt_ce3_fieldalias	CE-3: the flex→rigid alias drops $a \sim \text{List } a$ unexamined (a wrong-typed value)	err type variable a is rigid
rowgadt_ce3a_tuple	CE-3 class (a): the alias forms inside a tuple	err infinite type
rowgadt_ce3b_prealias	CE-3 class (b): the alias forms before the result unify	err infinite type
rowgadt_ce3c_let	CE-3 class (c): the alias forms through a <code>let</code>	err infinite type
rowgadt_shape_rebuild	the domain rule accepts the legitimate rebuild	clean
rowgadt_dup_rebuild	duplicate-label equation bodies rebuild exactly	clean
rowgadt_dup_fewer	the <i>pinned false reject</i> (do not weaken)	err cannot unify a with $\{x:\text{Int} \mid a\}$
rowgadt_fsm	the external FSM compiles and runs	run "open:42"
rowgadt_fsm_bad	illegal transition rejected at construction	err missing field closed
rowgadt_fsm_narrow	per-branch narrowing: a wrong-field read errs	err cannot be unified with $\{\text{broken}:a \mid b\}$
rowgadt_fsm_nested	the source's nested-chain match shape	run "open:42"
rowgadt_fsm_nested_bad	nested wrong-field read errs	err cannot be unified with $\{\text{broken}:a \mid b\}$
adtgaps	a missing possible arm is a compile error	err non-exhaustive case
refutpos	an impossible arm is refuted, not required	run 5
refutneg	a possible arm is NOT refuted	err non-exhaustive case
refutbare	a bare flexible index never licenses refutation	err non-exhaustive case: missing B

Re-homed probes the paper cites (not gate-registered; reproducible from `examples/research/` with the documented one-liner): the typed-representation experiment `probe A/probeA2` (clean) and `probeA_neg` (a wrong `compare` body is rejected, cannot unify number with `String`) – the trusted-body decomposition of Section 6.4; `gNeg_letcase.elm` – the retry’s completeness boundary (cannot unify `Bool` with `Int`); the duplicate-label battery `dup-hunt/e1–e7` (six must-error escape shapes and the one exact-rebuild accept, `e3_swap`, clean).

APPENDIX B: THE MECHANIZATION INVENTORY

One Lean project (`lean/`, `lake build clean`; 90 theorems, 3 axiom parameters, 0 `sorry/` `admit`). Cited by theorem name throughout.

file	theorems	content
<code>RowGadt.lean</code>	28	the row algebra; H1 (five theorems, Section 4.2); H2-a truncation; H2-b domain rule, exactness, over-approximation, alias/counterexample lemmas
<code>RowGadtEscape.lean</code>	4	the two hunt counterexamples mechanized: <code>H2b_no_quantify_refined_tail</code> , <code>H2b_wildcard_leak_shape</code> , plus the generalization lemmas
<code>Update.lean</code>	11	the update theorem and its insertion dual (Section 4.5)
<code>Typing.lean</code>	—	the constraint-based judgment skeleton: the rule set, with <code>Unifies/Captures</code> as axiom parameters (no theorems by design)
<code>TypingStore.lean</code>	29	store lemmas: weakening, monotonicity, the R-LET refined-tail exclusions, <code>tier_R_forces_escape</code> , <code>refinedTailVar_iff_insertionRejected</code>
<code>Preserve.lean</code>	18	record-operation preservation (Section 4.6): <code>sel/selDisch/upd</code> preserves, the two-tier soundness, the first-occurrence correspondence; <code>unifies_field_projection</code> as the third axiom parameter

The three axioms are parameters, not gaps in proofs: `Unifies/Captures` are the unifier as a parameter of the theory (Section 4.1), and `unifies_field_projection` is the one property the plain-selection preservation case needs of the abstract unifier. Every other theorem closes against the local Lean binary with no sorry.

REFERENCES

Provenance follows the companion survey (`docs/research/osier-related-work.md`), which is this bibliography’s source: `[M]` marks a work retrieved and read from its primary source (a URL is given wherever the survey records one), and `[SECOND-HAND]` marks a work carried exactly as the survey carries it, not retrieved. The list contains exactly the works this paper cites or names. Entries are alphabetical by author surname, or by system name where the paper cites a work by its system name (Koka, PureScript, OCaml, Links, GHC); the OCaml language, its release notes, its issue tracker, and its maintainers’ forum record are grouped under O. Where the survey records no title for a work, the entry gives the survey’s own descriptor of it; where it records no URL, none is given. A later bibliographic fill-in pass supplied the missing detail — published titles and canonical URLs where the survey carried a descriptor or no URL, and the thread/issue titles and dates for the forum and tracker entries. It read nothing the survey had not read, so the provenance markers are unchanged: a work the survey carries as `[SECOND-HAND]` stays `[SECOND-HAND]` even where its URL was located, because locating a work is not reading it.

- **Castagna & Peyrot (2025).** *Polymorphic Records for Dynamic Languages* — presence polymorphism over optional fields in a set-theoretic type system. OOPSLA’25 (Proc. ACM Program. Lang. 9, OOPSLA1, pp. 1464–1491). arXiv:2404.00338. <https://arxiv.org/abs/2404.00338> [M, lit]
- **Chen & Erwig (2016).** *Principal type inference for GADTs* — branch refinements represented by choice types $D\langle\Phi\rangle$, typing separated from reconciliation. POPL’16. Author’s version, retrieved 2026-10-06: https://web.engr.oregonstate.edu/~erwig/papers/TypeInfForGADTs_POPL16b.pdf (the ACM DL remains CAPTCHA-blocked). [M this pass]
- **Fan, Xu & Xie (2025).** *Practical Type Inference with Levels*. PLDI’25. <https://sec-r.github.io/papers/pldi25level.pdf> [M this pass]
- **Garrigue & Rémy (2012).** *Tracing ambiguity in GADT type inference* (3-page abstract). <http://gallium.inria.fr/~remy/gadts/Garrigue-Remy:gadts@abs2012.pdf> [M, lit]
- **Garrigue & Rémy (2013).** *Ambivalent Types for Principal Type Inference with GADTs* — $v(a)$. `M` introduces a rigid variable, quantified flexible at scope exit; scoped equations may not leak out of a branch. APLAS’13. <http://gallium.inria.fr/~remy/gadts/Garrigue-Remy:gadts@aplas2013.pdf> [M, lit]
- **Garrigue (2013).** OCaml compiler revision 13221 (2013-01-29), posted in `ocaml/ocaml` issue #5724: “At long last I have removed this restriction on the presence of polymorphic variants...” — the fix that lifted the 4.00-era restriction, shipped in 4.01. <https://github.com/ocaml/ocaml/issues/5724#issuecomment-472965878> [M — GitHub API]
- **Gaster & Jones (1996).** *A Polymorphic Type System for Extensible Records and Variants*. NOTTCS-TR-96-3. <http://web.cecs.pdx.edu/~mpj/pubs/96-3.pdf> [M this pass]
- **GHC.** User’s Guide, the `HasField` extension — class-triggered unification on record GADTs (`HasField unGadt (Gadt t) b` reduces by unifying $t \sim [v]$ and $b \sim \text{Maybe } v$). https://downloads.haskell.org/ghc/latest/docs/users_guide/exts/hasfield.html [M, lit]

- **hdgarrood (2019)**. Post in the PureScript discussion forum (Discourse), thread 707, “Thoughts on future additions of type-level features” (2019-04-03): “I think we should commit to not adding GADTs.” <https://discourse.purescript.org/t/707> [M – quote verified in the survey]
- **Hubers et al. (2025)**. *Abstracting Extensible Recursive Functions* – row-typed isomorphisms whose results vary with inputs via row constraints on return types. arXiv:2410.11742 (v2, July 2025; v1, Oct 2024, was titled *Extensible Recursive Functions, Algebraically*). <https://arxiv.org/abs/2410.11742> [M, lit]
- **Koka (Leijen 2014)**. *Koka: Programming with Row Polymorphic Effect Types* – the Koka language: effect rows with duplicate labels (from Leijen 2005); eliminations unify the effect row (`catch` unifies μ with $(\text{exn} \mid \mu')$), never refine it. MSFP’14 (EPTCS 153, pp. 100–126). arXiv:1406.2061. <https://arxiv.org/abs/1406.2061> [M, lit2]
- **Leijen (2005)**. *Extensible records with scoped labels*. TFP’05. <https://www.microsoft.com/en-us/research/wp-content/uploads/2016/02/scopedlabels.pdf> [M, lit]
- **Links**. Cooper, Lindley, Wadler & Yallop, *Links: Web Programming Without Tiers* – the Links language as published in this paper: HM type inference with row variables for records and variants; the concurrency calculus types functions by the messages they may accept ($\Gamma; C \vdash t : A$), not by effects. The paper contains no effect rows, no record update (its `update` is the SQL database operation), and no subtyping – the content earlier recorded in this entry (effect rows; subtyping via upcasts; record update derived remove-then-extend, or CPS + upcast) is attested only in the CORELINKS aside on the later “full version of Links”. FMCO’06, LNCS 4709, pp. 266–296, 2007. https://doi.org/10.1007/978-3-540-74792-5_12 (full text read: <https://homepages.inf.ed.ac.uk/slindley/papers/links-fmco06.pdf>) [M – retrieved and read 2026-10-07, author-page version “Submitted to FMCO ’06”; the effect-row/update/subtyping characterisation is credited to CORELINKS, not to this paper [M, lit2]]
- **Lindley & Cheney (2012)**. *Row-based Effect Types for Database Integration* (CORELINKS) – (label $\times$ presence $\times$ type) rows with distinct labels; insert/update/delete typed by quantifying a presence variable over the needed row. TLDI’12 (pp. 91–102). <https://homepages.inf.ed.ac.uk/slindley/papers/corelinks.pdf> [M, lit2]
- **Madsen, Starup & Lutze (2023)**. *Restrictable Variants: A Simple and Practical Alternative to Extensible Variants* (implemented in the Flix language) – refines a label-set index under `choose` while the record row index is only ever extended; GADTs named as “interesting future work”. ECOOP’23 (LIPIcs.ECOOP.2023.17). <https://drops.dagstuhl.de/storage/00lipics/lipics-vol263-ecoop2023/LIPIcs.ECOOP.2023.17/LIPIcs.ECOOP.2023.17.pdf> [M, lit]
- **OCaml (language)**. The ML-family language whose GADTs, locally abstract types (`type a.`), objects, and polymorphic variants are discussed in §1 and §7.4. Documentation: the OCaml manual, ocaml.org/manual/5.3/ (retrieved; it documents GADT equations on non-local abstract types and states no objects/poly-variants $\times$ GADTs restriction). The paper’s behavioural claims about the combination are attributed to the maintainer record: octachron (2024), [ocaml/ocaml #5724](https://ocaml.org/releases/4.01/notes/Changes), Garrigue (2013, rev 13221), and the OCaml 4.01.0 Changes. [M, lit + this pass]
- **OCaml 4.01.0 Changes (official release notes)**. “Propagate type information towards pattern-matching, even in the presence of polymorphic variants (discarding only information about possibly-present constructors). As a result, matching against absent constructors is no longer allowed for exact and fixed polymorphic variant types. (Jacques Garrigue)” <https://ocaml.org/releases/4.01/notes/Changes> [M – verified on the 4.01 branch raw source]

- **ocaml/ocaml issue #5724 (2012)**. “Interaction between GADTs and polymorphic variants” — “This is a documented limitation of GADTs: types cannot be refined if a pattern-matching contains polymorphic variant. This comes from the fact polymorphic variant pattern-matching typing is specified in the absence of type propagation and GADT type refinement requires type propagation.” Filed 2012-08 by yallop; closed as fixed 2015-12-11 by xavierleroy (fixed in 4.01.0+dev). <https://github.com/ocaml/ocaml/issues/5724> [M, lit]
- **octachron (OCaml maintainer) (2024)**. “Unable to refute impossible GADT pattern with polymorphic variants”, discuss.ocaml.org thread 13718, Jan 3 2024 — “the interaction of row variables and GADTs is not well specified”; “GADT equations cannot narrow a polymorphic variant constraint”; the per-field-object workaround. Cited in the text as [octachron 2024]. <https://discuss.ocaml.org/t/unable-to-refute-impossible-gadt-pattern-with-polymorphic-variants/13718> [M]
- **octachron (OCaml maintainer) (2024)**. “Does using object types in GADTs get you any more power?”, discuss.ocaml.org thread 14042, Feb 2024 — “GADTs are incompatible with object subtyping: they are always invariant with respect to their type parameters” (quoted in §7.4). <https://discuss.ocaml.org/t/14042> [M, lit]
- **O’Brien, Rémy & Scherer (2026)**. *Omnidirectional type inference for ML: principality any way*. arXiv:2511.10343, v2 May 2026. <https://arxiv.org/abs/2511.10343> [M this pass]
- **Ohori (1995)**. *A Polymorphic Record Calculus and its Compilation*. TOPLAS 17(6). <https://www.cs.tufts.edu/comp/150FP/archive/atsushi-ohori/record-calc.pdf> [M this pass]
- **Paszke & Xie (2023)**. *Infix-Extensible Record Types for Tabular Data* — a row-polymorphic record calculus with infix extension, for tabular data. TyDe’23 (Proc. 8th ACM SIGPLAN Int’l Workshop on Type-Driven Development, pp. 29–43). <https://dl.acm.org/doi/10.1145/3609027.3609406> [M this pass]
- **Peyton Jones, Vytiniotis, Weirich & Washburn (2006)**. *Simple Unification-based Type Inference for GADTs* — wobbly types: rigid vs wobbly annotation modifiers in the environment, for GADT inference. ICFP’06. <https://www.cs.tufts.edu/~nr/cs257/archive/simon-peyton-jones/gadt-icfp.pdf> [M this pass]
- **Pottier & Régis-Gianas (2006)**. *Stratified Type Inference for Generalized Algebraic Data Types* — two-pass type inference: shape inference (rigid information propagated through annotations), then constraint solving. POPL’06. <https://cambium.inria.fr/~fpottier/publis/pottier-regis-gianas-popl06.pdf> [M this pass]
- **PureScript (language)**. Documentation and issue tracker, retrieved 2026-10-06: row-polymorphic records documented first-class (rows are “an unordered collection of named types, with duplicates”; open rows via `| r`; kind `Row k`) — the type-system page, <https://github.com/purescript/documentation/blob/master/language/Types.md>; GADTs an unimplemented sketch (proposal #1448, “GADTs via Prism/Lens”, 2015, closed — <https://github.com/purescript/purescript/issues/1448>) and argued against by the core team (hdgarrood, thread 707, 2019 — <https://discourse.purescript.org/t/707>); existentials open since 2018 (#3492, “Support for existentials” — <https://github.com/purescript/purescript/issues/3492>). [M this pass]
- **Rémy (1994)**. *Type inference for records in a natural extension of ML*. UPenn technical report MS-CIS-90-73 (= RR-1431), Oct 1990 — the preprint of the 1994 MIT Press TAOOP chapter. <https://repository.upenn.edu/bitstreams/82820b99-74b8-4a44-bd03-0de4b3a91030/download> [M this pass]
- **Simonet & Pottier (2007)**. *A constraint-based approach to guarded algebraic data types (HMG(X))*. TOPLAS 29(1). <http://www.normalesup.org/~simonet/publis/simonet-pottier-hmg-toplas.pdf> [M this pass]

- **Spanò (2024).** *Flexible and Reversible Conversion between Extensible Records and Overloading Constraints for ML* — row polymorphism with overloading (reversible conversion between records and overloading constraints). arXiv:2406.11750. <https://arxiv.org/abs/2406.11750> [M this pass]
- **Toohey, Chen, Jamalzadeh & Xie (2026).** *Extensible Data Types with Ad-Hoc Polymorphism (Extended)*. Proc. ACM Program. Lang. 10, POPL, Article 20, Jan 2026 (38 pp extended version; the retrieved artifact). <https://xnning.github.io/papers/popl26extensible-appendix.pdf> (ACM DL landing, CAPTCHA-blocked: <https://dl.acm.org/doi/10.1145/3776662>) [M — full extended PDF retrieved 2026-10-05]
- **Vytiniotis et al. (2011).** *OutsideIn(X): Modular Type Inference with Local Assumptions* — implication constraints $\exists \tilde{\alpha}. (Q_{\text{given}} \leadsto C_{\text{wanted}})$ with touchable vs untouchable variables; the standard GADT-inference framework. J. Funct. Programming 21(4–5), pp. 333–412. <https://lirias.kuleuven.be/retrieve/237824> [M, lit]
- **Wand (1989).** *Type inference for record concatenation and multiple inheritance*. LICS’89. <https://www.cs.tufts.edu/comp/150FP/archive/mitch-wand/types-simple-objects.pdf> [M this pass]